Natural Language Generation from Structured Data

A THESIS

submitted by

SHREYAS SHETTY M

for the award of the degree

of

MASTER OF SCIENCE

(by Research)

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

INDIAN INSTITUTE OF TECHNOLOGY MADRAS

May 2018

THESIS CERTIFICATE

This is to certify that the thesis entitled **Natural Language Generation from Structured Data**, submitted by **Shreyas Shetty M**, to the Indian Institute of Technology, Madras, for the award of the degree of **Master of Science (by Research)**, is a bona fide record of the research work carried out by him under my supervision. The contents of this thesis, in full or in parts, have not been submitted to any other Institute or University for the award of any degree or diploma.

Dr. Mitesh. M. Khapra
Research Guide
Assistant Professor
Dept. of Computer Science and Engineering
IIT-Madras, 600 036

Place: Chennai

Date:

ACKNOWLEDGEMENTS

I would like to thank my advisor Prof. Mitesh Khapra for guiding and supporting me through the course of my MS. Working with him was a great learning experience, especially on the practical aspects of NLP and Deep Learning research. I hope to cultivate his level of attention to detail and enthusiasm for research. I also thank him for the opportunity to work closely for the first offering of the Deep Learning course. I got a better understanding of the material by working on setting assignments and other evaluations for the course. I am very grateful to Prof. Balaraman Ravindran for being very supportive throughout my MS. He sent me an invitation to attend the MSR Machine Learning Summer School in 2015. This was just at the end of my first semester and I had only had few interactions with him. I was still unsure about the kind of problems I wanted to pursue and the summer school was helpful in forming first impressions about the field of machine learning in general. I am also grateful to him for helping me out when I was looking for an advisor, which eventually led me to work with Prof. Mitesh Khapra. I thank Prof. P. Sreenivasa Kumar, Prof. Hema Murthy and Prof. Umesh S for serving on my GTC committee and giving useful feedback during the course of our interactions.

I worked closely with Preksha during my MS. This thesis would not be in its current form without her support and help. Thank you Preksha. I am also grateful to our collaborators from IBM IRL, Parag, Anirban and Karthik. I thank IBM Research India for providing access to compute resources which was very crucial for this work.

I am deeply indebted to Prof. Neeldhara Misra for supporting me with the Project Assistant position at the Department of CSA, IISc. She was extremely supportive as a mentor and it is only because of her that I developed an interest in theoretical aspects of Computer Science. Given that I had absolutely no background in computer science at the time, she was kind enough to teach me and help me out with getting my basics in place. It was from her that I had my first exposure to working on a research problem.

I would not be here today without the opportunity to work with her. I thank my friend Rajeev for introducing me to her.

Life at IIT Madras would not have been so much fun and memorable if not for the amazing set of friends. TCS lab has been a fun place and I will cherish the time spent there. Thank you Amit, Manas, Arinjita, Kanika, Suvradip, Monosij, Vijay, Rajesh and Dhannya for making TCS lab such an amazing place. I had a great time with my batch mates Siddharth, Shashank, Rajeev and Giriraj, especially the badminton sessions. Life at Tambi and IITM in general has been memorable because of the company of Manish, Nauman, Prathamesh, Dennis, Jom, Shashi, Saurabh, Sharmaji, Somesh, Karthik, Phani and others. Thanks guys for all the great memories. Thanks Tarun for being a wondeful co-TA for the NPTEL Machine Learning course.

My parents and my brother have been a constant source of inspiration. They have been extremely patient and supportive throughout. Their support and belief in me is what drives me to put in my best efforts. Not much would have been possible without them.

ABSTRACT

KEYWORDS: Deep Learning, Natural Language Generation, Recurrent Neural Networks, seq2seq models

Our interactions with computers is increasingly based on natural language. For instance, dialog systems help us book tickets and answer our queries. These systems give us the feeling of interacting with a human being and as a result makes our interactions easy and seem natural. A common component of these systems is the Natural Language Generation (NLG) system that generates human like text as a response to some input (query, source language text (machine translation), *etc.*).

Most standard NLG problems such as machine translation, question answering, dialog, summarization, *etc.*, process unstructured information (usually provided as raw textual input). However, there are NLG tasks where we are interested in processing structured information. A typical example of such a task is generating product descriptions from structured catalog tables. In this thesis, we focus on one such NLG task which involves rendering natural language descriptions from a structured table of facts. Specifically, we consider the problem of generating one line biographical descriptions of people given fact tables describing them (wikipedia infoboxes). Unlike other NLG tasks, the input here has a specific structure which can be exploited for building richer models. We experiment with a recently released WIKIBIO dataset for biographical descriptions in English. We have also created 2 new datasets for biographies in French and German.

NLG systems were traditionally built using rule-based systems. However, statistical methods have become increasingly popular in the NLG community. In the recent past, neural network based deep learning techniques have been very succesful in a wide variety of Natural Language Processing tasks. Most NLG problems can be formulated as a mapping from one sequence to another. For instance, in the context of machine

translation the source language sentences are mapped to the target language sentences. The sequence-to-sequence (seq2seq) paradigm based on a class of models known as Recurrent Neural Networks (RNN) has become a standard approach for such tasks. We propose models that are built on top of the seq2seq paradigm and encode the structural properties of the input data. Specifically we introduce 2 models, *viz.*, the flat model and the hierarchical model for biography generation. The flat model flattens the input infobox into an appropriate sequence and maps it to the corresponding biography description. On the other hand, the hierarchical model is based on the observation that there is an inherent hierarchy in the input data. Our experiments show that the proposed models give (i) 10% relative improvement (Flat model) and (ii) 18.8% relative improvement (Hierarchical model) over a recently proposed state of the art method. Our qualitative observations demonstrate the ability of the proposed models to capture interesting patterns in sentence generation and visualization of the intermediate outputs of the model provide justifications for the modelling choices.

Contents

ACKNOWLEDGEMENTS **i**

ABSTRACT **iii**

LIST OF TABLES **vii**

LIST OF FIGURES **ix**

ABBREVIATIONS **xi**

1 Introduction **1**

1.1 Problem definition 3

1.2 Contributions of the thesis 6

1.3 Outline of the thesis 6

2 Background **9**

2.1 Artificial Neural Networks 9

2.2 Training Neural Networks 11

2.3 Recurrent Neural Networks 12

2.4 LSTM and GRU 15

2.5 Sequence to Sequence models with Attention 17

2.5.1 RNN encoder-decoder model 18

2.5.2 Attention mechanism 19

2.6 Evaluation Measures 20

2.6.1 BLEU 21

2.6.2 NIST 22

2.6.3 ROUGE 23

3 Related Work **25**

3.1 Techniques 25

3.2 Datasets and Tasks 31
3.2.1 WEATHERGOV 31
3.2.2 WIKIBIO 32
3.3 Baseline approaches 33
3.3.1 Baseline 1: Coarse-to-Fine alignment model 34
3.3.2 Baseline 2: Table conditioned Neural Language Model . . . 37
3.4 Summary 43

4 Proposed Models for Biography Generation 45
4.1 Overview 45
4.2 Notation 46
4.3 Flat Model 46
4.3.1 Model Description 46
4.4 Hierarchy Model 50
4.4.1 Model Description 50
4.5 Implementation details 54
4.5.1 Dataset preprocessing 54
4.5.2 Neural network implementation 55
4.5.3 Post processing 59
4.6 Experiments 59
4.6.1 English 60
4.6.2 French 63
4.6.3 German 64
4.6.4 WEATHERGOV 64
4.7 Summary 65

5 Conclusions and Future work 67

LIST OF PAPERS BASED ON THESIS 77

List of Tables

2.1 BLEU computation example 22

3.1 WIKIBIO Dataset statistics 33

4.1 Sample infobox 47

4.2 Comparison of different models on the English WIKIBIO dataset . . 61

4.3 Comparison of different models on the French WIKIBIO dataset . . 64

4.4 Comparison of different models on the German WIKIBIO dataset . . 64

4.5 Comparison of different models on the WEATHERGOV dataset . . . 65

List of Figures

1.1 Product information from ecommerce website 4

1.2 Wikipedia infobox of Vincente Mijares Saldivar. The corresponding description from Wikipedia is: *"Vicente Mijares Saldivar (born June 14, 1954 in Gómez Palacio, Durango, Mexico) is an undefeated Mexican professional boxer in the Lightweight division"* 5

2.1 Sigmoid neuron 9

2.2 Feed-forward neural network 10

2.3 Backpropagation example 12

2.4 Recurrent Networks 13

2.5 LSTM Cell 15

2.6 GRU Cell 16

2.7 Encoder Decoder model 19

3.1 Sample scenario from WEATHERGOV dataset 32

3.2 Infobox for Prof. V. Balakrishnan 33

3.3 Coarse-to-fine grain alignment model architecture 34

3.4 Infobox for illustrating local conditioning 40

3.5 Local conditioning illustration 40

4.1 Flat model block diagram 48

4.2 Hierarchy model block diagram 50

4.3 Vicente Mijares - Attention Visualization 61

4.4 Masayoshi Tomizuka - Attention Visualization 62

5.1 Leonarda da Vinci English and French infobox 68

ABBREVIATIONS

NLP	Natural Language Processing
NLG	Natural Language Generation
seq2seq	Sequence to Sequence models
RNN	Recurrent Neural Network
LSTM	Long Short Term Memory
GRU	Gated Recurrent Unit

Chapter 1

Introduction

Humans and animals have the unique capability to learn and adapt to their environment. This ability to demonstrate intelligent behaviour has been crucial for the survival of our species. Humans in the course of evolution have developed highly sophisticated mechanisms to deal with everyday scenarios. We have moved from basic hand gestures for communication to highly advanced systems of spoken and written language. In fact, development of language is one of the important factors that has lead to our incredible progress from cave dwellers to modern-day humans.

Intelligence and language are closely related. Philosophers have long pondered over the question "What is intelligence?". Philosophical ramifications aside, we have been interested in building machines that can interact with the surrounding environment and perform certain desired actions. The famous mathematician Alan Turing considered the question "Can machines think?" (Turing (1950)) and made an attempt to formally define the notion of intelligence. He proposed the famous Turing test to ascertain if a machine is intelligent. The test involves a human evaluating a conversation between a human and a machine designed to generate natural language responses. This work of Turing was followed with subsequent work by Claude Shannon who described chess playing as a problem of search and at around the same time Issac Asimov published the three laws of robotics. These efforts lead to the birth of the field of Artificial Intelligence (AI). Early efforts in AI focussed on building game playing systems and then branched out to tackle broader class of problems. Traditionally AI research has been based on rule-based and symbolic systems that operate on high-level representations of problems and logic. Machine learning, on the other hand, tries to build systems which can "learn" from experience. Depending on the type of experience that the system gets, machine learning is subdivided into supervised learning (labelled training examples), unsupervised learning (unlabelled data) and reinforcement learning (learning from interaction). Machine learning is widely applied in a variety of problems ranging from

images, video, text and speech. Recently, machine learning has shown promise in our goal of building truly intelligent systems.

As mentioned above, to pass the Turing test, machines need to be able to process and make sense of natural language. Natural Language Processing (NLP) is a branch of artificial intelligence that focusses on the interactions between human languages and computers. Problems in NLP involve design of efficient algorithms for understanding, analyzing and generating human (natural) language and can be categorized under Natural Language Understanding (NLU) and Natural Language Generation (NLG). NLP is the foundation of modern systems for machine translation, automatic spelling correction, auto-suggest *etc.* Early work in NLP focussed on rule-based systems and relied on work in linguistics (Chomsky (1975)). Since the late 1980's the emphasis in NLP has been on statistical techniques (Manning and Schütze (1999)) and has led to a lot of work at the intersection of machine learning and NLP.

Natural Language Understanding (NLU) is concerned with understanding and comprehending input text. NLU applies from basic scenarios such as a robot understanding simple short text commands, to systems which try to comprehend paragraphs of text and perform some action (for example, answer questions based on some input text). NLU is considered to be an AI-complete problem, in the sense that the difficulty of NLU is equivalent to solving the problem of making computers as intelligent as humans.

Natural Language Generation (NLG) on the other hand focuses on building systems for generating understandable and coherent text in human languages (for example English). NLG typically forms the front-end of systems that interact with humans. For instance, a chatbot generates conversations to engage a user toward a desired end goal (say booking a flight ticket). Popular applications of NLG include machine translation, dialog systems and automatic summarization. Machine Translation systems are designed to generate text in the target language (for example English) by processing the text in the source language (for example French). The challenge in machine translation is to account for different vocabulary and potentially different sentence construction rules across languages. Dialog systems interact with users by generating responses to user queries/conversations. It is important to note that keeping track of long range information is hard, also human conversations in general involve multiple context switches.

The system needs to keep track of the context of the conversation and generate appropriate responses in natural language. Summarization systems are categorized under two types, abstractive and extractive summarization. In extractive summarization the model chooses appropriate sentences from the source document, abstractive summarization on the other hand generates a shorter summary by rephrasing the contents of the source document. Abstractive summarization systems involve an NLG aspect, where the model needs to process the source document and generate relevant summary. In all the above mentioned examples, the system has unstructured natural language text as input and the task is to generate corresponding translation/response/summary. In this thesis, we focus on the specific problem of NLG with input as structured data as defined in the next section.

1.1 Problem definition

The advent of internet has led to a lot of data being generated and collected from a number of users. A significant amount of research effort has gone into building systems to handle, process and make sense of large amounts of data. For example, database systems are explicitly designed to handle and process (large) tabular data. Machine learning algorithms are usually used to derive insights from data. Among other things, these insights drive business for corporations and improve the end-user experience for customers.

Most of our interaction with computers is usually in the form of text and visuals (graphical interfaces). Computers are designed to deal with structured data (for example, web forms), humans on the other hand prefer to read and interpret information as text (natural language descriptions). Natural Language Generation (NLG) systems have become crucial in serving as a bridge for interactions between humans and computers. For example, machine translation systems have unlocked access to text written in different languages to a wider audience, dialog systems provide interfaces for humans to interact with machines for booking hotel reservations, flight tickets *etc.* However, these systems manipulate data that is usually unstructured (for example, text in source language to text in target language).

As mentioned before, large amounts of data is usually stored as structured information. Systems for NLG from structured data let us have user friendly access to structured data. For example,

- Automatic generation of profile summaries (from profile descriptions on websites such as Facebook) can be used for providing short descriptions of individual users.

- Hotel/resort descriptions can be generated based on tabulated information available from travel booking websites (for example, Expedia).

- E-commerce websites can generate product descriptions using the catalog tables of products listed on the site, helping users make better product purchase decisions. For example consider the following product description table for Apple iPhone 6s on Amazon. A candidate description for the above product is as follows:

Apple iPhone 6s Mobile Phone Information

Colour: **Rose Gold**

Technical Details

OS	iOS
RAM	2 GB
Item Weight	145 g
Product Dimensions	6.7 x 0.7 x 13.8 cm
Item model number	iPhone 6s
Wireless communication technologies	Bluetooth, WiFi Hotspot
Connectivity technologies	WLAN Wi-Fi 802.11 a/b/g/n/ac, dual-band, hotspot Bluetooth v4.2, A2DP, LE GPS Yes, with A-GPS, GLONASS NFC Yes (Apple Pay only) Radio No USB v2.0, reversible connector
Special features	Accelerometer, gyro, proximity, compass
Other camera features	5 MP
Form factor	Touchscreen Phone
Weight	145 Grams
Colour	Rose Gold
Battery Power Rating	1715
Whats in the box	Handset, Charger, EarPods, Lightning to USB Cable and Documentation

Figure 1.1: Product information from ecommerce website

"The Apple iPhone 6s is a lightweight, slim, touchscreen mobile phone that runs on iOS. It features a 5MP camera and 2 GB RAM. It comes in Rose Gold color."

Early efforts in NLG from structured data included generating weather forecasts (Liang *et al.* (2009), Mei *et al.* (2016*b*)) from automatically tabulated meteorological measurements, generating commentaries on Robocup matches (Chen and Mooney (2008)) based on the game state *etc.* These problems serve as prototypes for building

systems to describe structured data in natural language. We specifically consider the biography generation problem (Lebret *et al.* (2016))that is significantly more challenging than previous NLG from structured data tasks in terms of scale and the richness of the domains under consideration. The task is to generate a biography of a person based on their Wikipedia infoboxes. Specifically, an infobox is a table of facts describing a person and the first sentence of the corresponding Wikipedia article is the target description (biography) of the person.

Figure 1.2 shows an infobox and a corresponding description from Wikipedia. The biography generation task is to take the infobox as input and generate the one sentence description.

Vicente Mijares	
Statistics	
Real name	Vicente Mijares Saldivar
Nickname(s)	Orgullo de Durango
Weight(s)	Lightweight
Height	5 ft 10 in (178 cm)
Reach	73 in (183 cm)
Nationality	Mexican
Born	June 14, 1954 (age 63) Gómez Palacio, Durango, Mexico
Stance	Orthodox
Boxing record	
Total fights	37
Wins	29
Wins by KO	13
Losses	7
Draws	1
No contests	0

Figure 1.2: Wikipedia infobox of Vincente Mijares Saldivar. The corresponding description from Wikipedia is: *"Vicente Mijares Saldivar (born June 14, 1954 in Gómez Palacio, Durango, Mexico) is an undefeated Mexican professional boxer in the Lightweight division"*

The problem requires us to describe the facts presented in the infobox in a single sentence. Hence, the model needs to decide the important facts from the infobox, the order in which the facts need to be presented and finally use appropriate words to describe the infobox. Another peculiar characteristic of the problem is that a lot of information gets copied from the infobox onto the description (for example, name of the person). Models for biography generation need to take cognizance of these observations for effective generation of sentences.

Lebret *et al.* (2016) released the WIKIBIO [1] dataset for the biography generation

[1] https://github.com/DavidGrangier/wikipedia-biography-dataset

problem. Unlike other previously used datasets (WEATHERGOV, ROBOCUP, *etc.*) for similar NLG from structured data tasks, the WIKIBIO dataset consists of a diverse set of input fact tables (infoboxes) and contains order of magnitude more examples. We experiment with the WIKIBIO dataset and two newly created datasets for French and German biographies.

1.2 Contributions of the thesis

In this thesis, we propose 2 models which adapt and improve the neural encode-attend-decode framework for our task. In particular, we propose a neural network based encoder for structured data containing a hierarchical organization of facts. We experiment with two standard datasets (WEATHERGOV, WIKIBIO) for this task and demonstrate that our model gives state of the art results. Further, we have curated 2 new datasets for biographies in German and French. We do qualitative analysis of the generated outputs and visualize the intermediate outputs of the model thereby justifying out modeling choices. We hope that our work provides a starting point for approaching other related problems in NLG from structured data.

1.3 Outline of the thesis

Chapter 2 presents an overview of neural networks, recurrent neural networks and sequence to sequence models with attention. This background will help understand our proposed models. Metrics for the evaluation of generated sentences are discussed at the end. Chapter 3 reviews techniques for natural language generation. We also discuss two datasets and related tasks that have been used in the literature for NLG from structured data. This chapter ends with a discussion of two state of the art papers that we will use as baselines for comparison with our work.

In chapter 4 we describe our proposed models. We build intuitions for our modelling choices and explain the flat model and hierarchy model followed by a discussion of the results and our observations.

Finally, chapter 5 discusses the conclusions drawn from this thesis. We end with a discussion on directions for future research.

Chapter 2

Background

Artificial Neural Networks (ANNs) are biologically inspired models for machine learning. ANNs have been applied to a wide variety of problems including speech processing, image processing, natural language processing *etc.,* with phenomenal success. In this chapter, we start with an introduction to artificial neural networks and discuss various components of the network, followed by an overview of algorithms used for training them. We then discuss Recurrent Neural Networks (RNNs) which play an important role in various NLP problems. Specifically, we describe how they are used for modelling problems which require translating one sequence to another. Finally, we describe the evaluation metrics used to report our results.

2.1 Artificial Neural Networks

Biological neurons are the primary computational units in our brains. They receive, process and transmit information using electrical and chemical signals in our brain. Artificial neurons are simplistic models of biological neurons. Similar to a biological neuron, an artificial neuron takes in a bunch of numbers as input, computes some (non-linear) function of the input and outputs another number. The function used by the neuron is referred to as the activation function. Figure 2.1 shows a sigmoid neuron which takes in 4 inputs and generates a single number as an output by applying the sigmoid function on a linear combination of its inputs.

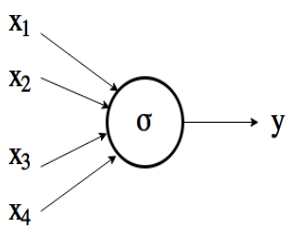

Figure 2.1: Sigmoid neuron

The human brain is a massive network of interconnected neurons. Artificial neural networks implement this idea using an interconnected network of simple artificial neurons (similar to the one shown in figure 2.1). The most basic family of neural network models is the feedforward neural network (also referred to as multi-layer perceptron or multi-layer feedforward neural network).

A feedforward neural network consists of neurons arranged in layers. Each layer comprises of multiple neurons. The output of a given layer of neurons feeds into the next layer of neurons. The first layer of neurons is usually referred to as the input layer and the final layer of neurons is referred to as the output layer. The layers in between are referred to as hidden layers. Figure 2.2 shows a feedforward neural network which takes in as input a vector of length 4 and outputs a single number. The network has a single hidden layer. Notice that all the transformations (arrows) go from left to right and hence the name "feedforward network".

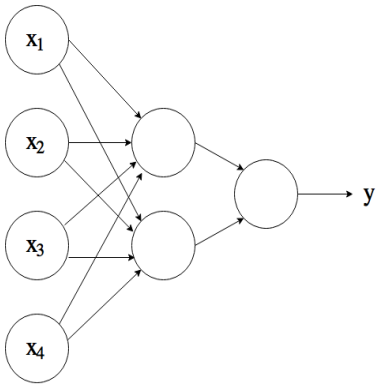

Figure 2.2: Feed-forward neural network

Formally, given an input $x \in \mathbb{R}^d$, a neural network with L layers computes a non-linear function by applying a series of transformations. We denote the number of neurons in layer i ($0 < i < L$) by n_i ($n_0 = d$). The output layer contains k neurons. The weight and bias between layers $i-1$ and i ($0 < i < L$) are $W_i \in \mathbb{R}^{n_i \times n_{i-1}}$ and $b_i \in \mathbb{R}^{n_i}$ respectively. The weight and bias between the last hidden layer and the output layer is denoted by $W_L \in \mathbb{R}^{n \times k}$ and $b_L \in \mathbb{R}^k$. Each layer implements a non-linear transformation of the input to the corresponding layer.

Mathematically, feedforward networks are a powerful family of function approximators. Hornik *et al.* (1989), Cybenko (1989) show that a feedforward network with a single hidden layer with a sufficient number of neurons can approximate any function

to an arbitrary level of accuracy. However, in practice, feedforward networks are implemented with multiple hidden layers with fewer neurons in each layer. Each layer learns a progressively higher level representation of the input, thus aiding the final task (classification or regression). Deep feedforward networks are discussed in detail in Chapter 6 in the book by Goodfellow *et al.* (2016).

2.2 Training Neural Networks

Once we have chosen an appropriate architecture (number of layers, neurons in each layer, activation functions *etc.*), we need to find the network parameters (weights and biases) that enable the model to perform well on the chosen task. We define a loss function (L) to quantify the quality of a set of parameters (θ). One standard choice for the loss function is the squared error loss

$$L(\theta) = \frac{1}{2}\sum_{i=1}^{m}(y_i - f_\theta(x_i))^2$$

where m is the number of training examples, y_i is the ground truth output for example x_i and $f_\theta(x_i)$ is the output predicted by the model. Given this formulation, the learning problem is cast as an optimization problem whose objective is to find the values of parameters that minimize the loss function.

Gradient Descent is used for minimizing the loss function and involves computing the gradient of the loss function with respect to all the parameters of the network and updating the parameters slightly in the direction of decreasing gradient.

Formally,

$$\theta_{t+1} \leftarrow \theta_t - \eta \nabla_\theta L(\theta_t) \tag{2.1}$$

where θ_t is the vector of current parameters, θ_{t+1} is the updated vector of parameters, η is referred to as the learning rate and $\nabla_\theta L(\theta_t)$ is the gradient of the loss function with respect to the network parameters. Researchers have developed several variants of the basic gradient descent algorithm that have better convergence properties and that have been empirically found to perform well. Ruder (2016) provides a thorough discussion

on gradient descent and related optimization algorithms. For our experiments we use the Adam Kingma and Ba (2015) optimizer. The method essentially computes adaptive learning rates for each parameter.

For performing gradient descent updates given by equation (2.1), we need to compute the gradient of the loss function with respect to all the parameters of the network. For example the toy network shown in Figure 2.3 the network has three parameters a, b and c respectively and we need to compute the derivates of the loss function with respect to all the 3 parameters i.e. the gradient $\nabla_\theta L$ requires computation of 3 partial derivatives.

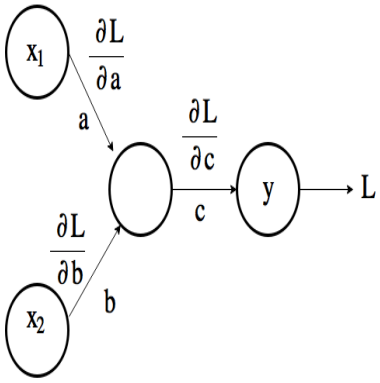

Figure 2.3: Backpropagation example

A naive implementation of the chain rule of derivatives to compute the gradient would be highly inefficient, especially for the networks that are of practical interest. Rumelhart *et al.* (1986) introduced an efficient algorithm for computing the gradients of the network parameters, known as the backpropagation algorithm. Backpropagation is used extensively in the training of modern deep networks. Deep Learning frameworks (TensorFlow, Torch, PyTorch, MXNet, CNTK etc.) provide implementations of backpropagation and optimization algorithms (for example SGD, Adam). We use TensorFlow for all our experiments.

2.3 Recurrent Neural Networks

Feed forward neural networks as discussed in the previous section, are useful in scenarios where the length of the input is fixed. For example, in the case of image classification all the input images are of the same dimension. However, there are many problem

settings where we would like to process variable length inputs. For instance, say we want to process a movie review and classify it as a positive/negative review. The length of the input (number of words/characters) is not fixed and the words in the input are not independent of each other. Recurrent neural networks (RNN) are a class of models that let us process such variable length input sequences. As shown in Figure 2.4 RNNs have recurrent connections (current state feeds into the next state computation). The input at time step t is denoted by x_t and the state of the RNN at the corresponding time step is denoted by h_t. Note that the current state h_t is a function of both the current input x_t and the previous state h_{t-1}. In effect, the state h_t acts as an aggregated representation of all the inputs that have been read so far. The fundamental idea underlying recurrent neural networks is that of parameter sharing. This enables us to apply the model on inputs of variable length and generalize across them. In particular, notice that the Recurrent Neural Network (RNN) shares the same weights across all time steps.

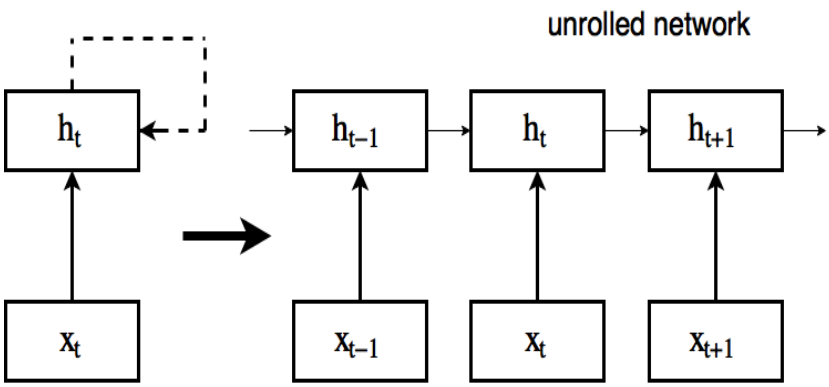

Figure 2.4: Recurrent Networks

More formally, an RNN operates on a variable length input sequence $\mathbf{x} = (x_1, \ldots, x_T)$, where $x_i \in \mathbb{R}^n$ and maintains a hidden state $h_t \in \mathbb{R}^d$ and has an optional output $y_t \in \mathbb{R}^k$ (k classes) at each time step. The hidden state of the RNN at timestep t is computed as

$$h_t = f(h_{t-1}, x_t) \tag{2.2}$$

where f is some non-linear function (depending on the type of the RNN cell used). For instance,

$$h_t = tanh(Wh_{t-1} + Ux_t)$$

where $W \in \mathbb{R}^{d \times d}$ and $U \in \mathbb{R}^{d \times n}$ are the parameters of the RNN. We collectively refer

to the parameters as θ.

We can also have an additional RNN that processes the sequence in the reverse order. In such a scenario, the state at time-step t is taken to be the concatenation of the states of the two RNNs. This variant is known as the bidirectional RNN (Schuster *et al.* (1997)).

An RNN can be used to learn a probability distribution of a sequence of values by training the model to predict the next value in the sequence. In this setup the output of the RNN at each time step (y_t) is the conditional probability distribution

$$y_t = g(h_t) = p(x_t|x_{t-1}, \ldots, x_1) \tag{2.3}$$

here y_t is computed using the softmax function ($g(h_t)$) over the set of possible outputs. The output probabilities from equation 2.3 can be combined to obtain the probability of the entire sequence $\mathbf{x} = (x_1, \ldots, x_T)$ as follows,

$$p(\mathbf{x}) = \prod_{t=2}^{T} p(x_t|x_{t-1}, \ldots, x_1) \tag{2.4}$$

The distribution 2.4 can be used to sample new sequences. We could initialize the sequence with some value and then generate one value at a time.

Recurrent neural networks are typically trained with SGD (or some variant such as Adam). Backpropagation is done by unrolling the RNN as depicted in Figure 2.4. RNNs suffer from the problem of vanishing and exploding gradients. Essentially, the recurrence in the RNN equations leads to a product of gradients in the backprop equations. Depending on the magnitude of the terms, the gradients of interest can either vanish or explode. This issue is explored in detail in Pascanu *et al.* (2013). Typically, the gradient explosion problem is handled using gradient clipping. Essentially, whenever the gradient exceeds a certain magnitude, its value is set to a certain threshold. Vanishing gradients on the other hand are tricky to handle and researchers have designed specialized RNN cells which circumvent the issue. Popular variants of RNN include LSTM (Hochreiter and Schmidhuber (1997)) and GRU (Cho *et al.* (2014)) cells as discussed in the next section.

2.4 LSTM and GRU

Long Short Term Memory (LSTM) network (Hochreiter and Schmidhuber (1997)) is a variant of the standard RNN. As discussed in Section 2.3, RNNs have the problem of vanishing and exploding gradients which limit the number of time-steps that can be tracked. LSTMs were proposed to handle this very issue, and are able to keep track of long range dependencies.

As mentioned eariler, RNNs transform a variable length input sequence into a vector of fixed size. With every new input in the sequence, the RNN updates (morphs) its state. This can potentially overwrite important information that was already stored in the RNN state vector, making it difficuly to keep track of long range dependencies. LSTMs were proposed to address these issues. LSTMs follow a structure similar to RNNs (i.e. same computation applied across steps) but the inner cell is composed of smaller substructures. Specifically, there are four substructures which interact to facilitate the LSTM cell to keep track of dependencies and subvert the vanishing gradient problem.

Figure 2.5 [1] depicts an LSTM cell. As with the standard RNN the LSTM cell maintains a "cell state" (c_t) which keeps track of information read from the input. LSTMs have 3 additional structures (referred to as gates) which have the ability to add or remove information from the cell state. The three gates are the "forget gate" (f_t), "input gate" (i_t) and the "output gate" (o_t) respectively.

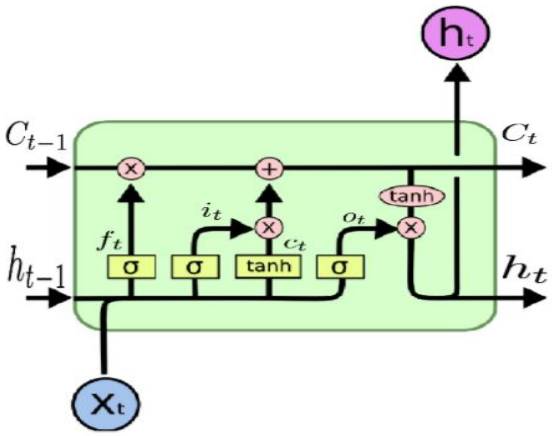

Figure 2.5: LSTM Cell

The following set of equations specify the exact computations implemented at each

[1] Image from Chris Olah's blog - https://colah.github.io/posts/2015-08-Understanding-LSTMs/

of the gates. [2]

$$
\begin{aligned}
f_t &= \sigma(W_f[h_{t-1}, x_t] + b_f) \\
i_t &= \sigma(W_i[h_{t-1}, x_t] + b_i) \\
\tilde{c}_t &= tanh(W_c[h_{t-1}, x_t] + b_c) \\
c_t &= f_t * c_{t-1} + i_t * \tilde{c}_t
\end{aligned} \tag{2.5}
$$

The forget gate decides the information that needs to be retained from the previous cell state and the information that needs to be discarded. The input gate then decides the new information (based on the current input) needs to be added to the cell state. Finally, the output gate decides what needs to be output from the cell. Selective read/write with the help of these gates ensures that important information from the earlier states do not get morphed unless the input forces the gates to forget past information or new inputs lead to overwriting past memory. An important feature of the LSTM cell is that it computes c_t (equation (2.5)) by a direct addition of c_{t-1} and a non-linearly transformed version of the past state. This provides a direct linear connection between c_t and c_{t-1} providing shortcut connections for information to flow. Such similar ideas have been used in powerful models for computer vision such as ResNets.

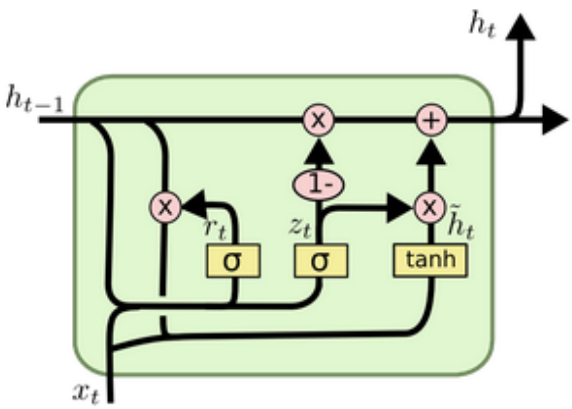

Figure 2.6: GRU Cell

Gated Recurrent Unit (GRU) Cho *et al.* (2014) is a variant of the LSTM cell. Figure

[2]The equations slightly vary across implementations, but the central ideas remain the same. $[h_{t-1}, x_t]$ denotes concatenation of the two vectors, and $f_t * c_{t-1}$ denotes element-wise product.

2.6 [3] depicts the GRU cell. The GRU cell combines forget and input gates into an "update gate". The cell state C_t and hidden state (h_t) from the LSTM cell are combined to have an effective state (h_t). The following set of equations describe the computations implemented in a GRU cell.

$$
\begin{aligned}
z_t &= \sigma(W_z[h_{t-1}, x_t]) \\
r_t &= \sigma(W_r[h_{t-1}, x_t]) \\
\tilde{h}_t &= tanh(W[r_t * h_{t-1}, x_t]) \\
h_t &= (1 - z_t) * h_{t-1} + z_t * \tilde{h}_t
\end{aligned}
$$

The update gate (z_t) captures how much from the past state needs to be carried onto the next state. The reset gate (r_t) is used to compute a temporary new memory content ($\tilde{h}_t$) which weighs the information from the past state and the current input. The GRU cell is a relatively simpler model compared to the LSTM cell and have fewer gating structures. Intuitively, if the reset gate is close to 0, the cell ignores the previous state allowing it to drop irrelevant information from the past. If the update gate z_t is close to 1, the cell can copy information through time steps leading to less vanishing gradients. Whenever there is short term dependencies the reset gates are active and in the presence of long term dependencies the update gates are active. In vanilla RNN the error has to backpropagate through all intermediate states, however in an LSTM and GRU cell the presence of gates creates shortcut connections that are adaptive which lets information to flow through larger timesteps and circumvent the vanishing gradient issue. Both GRU and LSTM can be used as bidirectional RNNs. We found that for our experiments bidirectional GRU cells performed better.

2.5 Sequence to Sequence models with Attention

In section 2.3 we saw an approach to use RNNs to generate sequences by learning a probability distribution over sequences. Many interesting problems can be cast as

[3] Image from Chris Olah's blog - https://colah.github.io/posts/2015-08-Understanding-LSTMs/

mapping sequences to sequences. For instance, the problem of Machine Translation can be viewed as that of mapping a sequence of words in the source language to a sequence of words in the target language. Neural models for such problems, as proposed in Cho *et al.* (2014), Sutskever *et al.* (2014) and Kalchbrenner and Blunsom (2013), use two RNNs known as the encoder and decoder respectively. The idea is to map the input sequence (of variable length) into a fixed length vector representation (encoder) and use this representation to generate the target sequence (decoder) using another sequence model (RNN/LSTM/GRU etc.). We start with a description of the RNN encoder-decoder model. It has been empirically observed that the restriction of mapping a variable length sequence to a fixed length vector leads to poor performance on sequence tasks. We then discuss attention mechanism Bahdanau *et al.* (2014) that was proposed to address this limitation.

2.5.1 RNN encoder-decoder model

The RNN encoder-decoder maps a variable length input sequence $\mathbf{x} = (x_1, \ldots, x_T)$ to a variable length output sequence $\mathbf{y} = (y_1, \ldots, y_{T'})$. The two sequences need not necessarily be of the same length (i.e. T and T' need not be equal).

The encoder is usually an RNN that processes the input $\mathbf{x}$ and updates its hidden state using equation (2.2). The hidden state of the RNN at the end of the sequence $h_T(= \mathbf{c})$, is treated as a fixed length summary of the input sequence ($\mathbf{x}$). The decoder is another RNN trained to generate the output sequence ($\mathbf{y}$). Figure 2.7 depicts the encoder-decoder model.

The hidden state (s_t) and output of the decoder RNN are given by,

$$s_t = f(s_{t-1}, y_{t-1}, \mathbf{c}) \tag{2.6}$$

$$p(y_t|y_{t-1}, \ldots, y_1, \mathbf{x}) = g(s_t, y_{t-1}, \mathbf{c}) \tag{2.7}$$

Notice that, in addition to s_{t-1}, s_t and y_{t-1}, equations (2.6), (2.7) are also dependent on the final state of the encoder ($\mathbf{c}$).

The network is jointly trained to maximize the conditional log-likelihood $p_{\boldsymbol{\theta}}(\mathbf{y}|\mathbf{x})$,

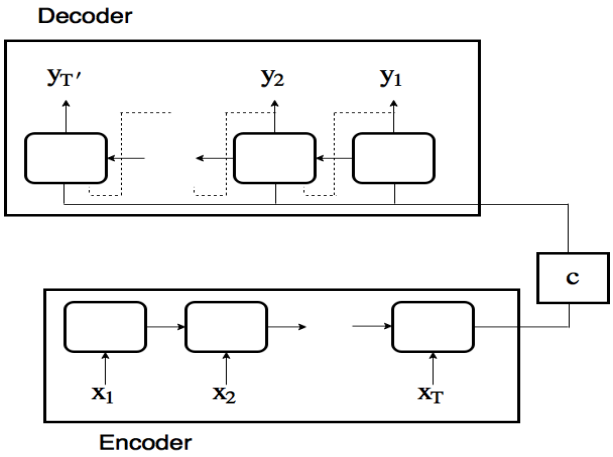

Figure 2.7: Encoder Decoder model

where $\boldsymbol{\theta}$ are the parameters of the network. The model once trained can be used to generate the output sequence $\mathbf{y}$ given an input sequence or can be used to score a given pair of source-target sequence$(\mathbf{x}, \mathbf{y})$. In our application, we use the encoder-decoder model to generate the output sequence conditioned on the input sequence. It was observed by Bahdanau *et al.* (2014) that as the source sequence length increases the performance of the encoder-decoder model (BLEU score) degrades. The degradation in performance is because the fixed length vector is insufficient to capture the information from longer input sequences. In the context of machine translation, Sutskever *et al.* (2014) found that reversing the input sequence helped the optimization problem by introducing short term dependencies. On the other hand, Bahdanau *et al.* (2014) proposed attention mechanism to handle such long input sequences, as discussed in the next section.

2.5.2 Attention mechanism

In the encoder-decoder model described in 2.5.1 a fixed length summary $\mathbf{c}$ (context) of the input sequence was used to feed the decoder RNN. Bahdanau *et al.* (2014) propose using a distinct context vector $\mathbf{c_t}$ for each word in the output y_t (or for each time step of the decoder). They define the probability in equation (2.7) as

$$p(y_t|y_{t-1}, \ldots, y_1, \mathbf{x}) = g(s_t, y_{t-1}, \mathbf{c_t})$$

where s_t is the decoder RNN state at time-step t (we use s_t and h_t to denote encoder RNN state and decoder RNN state respectively). Note that the context $\mathbf{c_t}$ is now different for each time step t. The hidden state of the decoder RNN is computed using

$$s_t = g(s_{t-1}, y_{t-1}, \mathbf{c_t})$$

The context vector $\mathbf{c_t}$ is function of all the input states generated by the encoder RNN $(h_1, \ldots, h_T)$. These encoder states computed at every time step t, capture the input information at time step t as well as the information from the surrounding words at time step t. The final context vector fed to the decoder is the weighted sum of the encoder states

$$\mathbf{c_t} = \sum_{j=1}^{T} \alpha_{tj} h_j$$

The weight α_{tj} for each encoder state h_j is computed as

$$\alpha_{tj} = \frac{\exp(e_{tj})}{\sum_{k=1}^{T} \exp(e_{tk})}$$

with $e_{tj} = a(s_{t-1}, h_j)$, and $a()$ is parameterized by a feedforward neural network. Bahdanau *et al.* (2014) refer to this as the alignment model that scores the alignment between the inputs around position j with the output at time-step t. Note that α_{tj} forms a probability distribution for each time-step t. The context vector $\mathbf{c_t}$ can then be viewed as the expected input context at time-step t. In effect, at each decoder time-step t, the decoder decides the relevant parts of the input and then generates the word at the output. This feature of the decoder to *focus* on relevant information in the input is referred to as "attention mechanism".

2.6 Evaluation Measures

So far we have discussed approaches to building models for sequence-to-sequence tasks. Traditional methods of evaluation (precision, accuracy, F1 etc.,) do not apply to our setup. Further, since we are evaluating natural language text, our evaluation needs to account for certain other features of natural language (for example, fluency).

Researchers have come up with multiple metrics to evaluate the quality of machine generated text. The popular evaluation metrics include BLEU (Papineni *et al.* (2002)), NIST (Doddington (2002)) and ROUGE (Lin (2004)). These metrics have been proposed in the context of evaluation of Machine translation systems (BLEU, NIST) and summarization systems (ROUGE). We evaluate our models using all these metrics and observe that our models perform well with respect to all these metrics. We now briefly describe the BLEU evaluation metric. The other two metrics make similar computations and we mention the differences towards the end of the chapter.

2.6.1 BLEU

BLEU (BiLingual Evaluation Understudy) uses a modified form of precision as a basis for comparison between a piece of machine generated text (candidate) and one or more reference texts. The computation involves counting the number of n-grams in the candidate that match some n-grams in the reference. Note that these comparisons are agnostic to the word order. These n-gram counts are modified to prevent rewarding candidates that contain an abundance of relevant words.

We now demonstrate the computation of BLEU score (BLEU-4) on outputs from two systems given a reference sentence [4]. The computation proceeds as follows, (i) compute n-gram precisions for 1 to 4 grams (ii) account for too short candidates (by adding brevity penalty). The overall formula for computing the BLEU-4 score is given by

$$BLEU = \min\Big(1, \frac{output - length}{reference - length}\Big)\Big(\prod_{i=1}^{4} precision_i\Big)^{\frac{1}{4}} \tag{2.8}$$

Reference: Israeli officials are responsible for airport security

System A: Israeli officials responsibility of airport safety

System B: airport security Israeli officials are responsible

According to the BLEU metric, system B performs better. Papineni *et al.* (2002) perform human evaluations to demonstrate that BLEU scores correlate well with human judgements. For computing BLEU-4 score on the entire corpus equation (2.8) is

[4]Example borrowed from the book "Statistical Machine Translation" by Phillip Koehn

Metric	System A	System B
precision (1-gram)	3/6	6/6
precision (2-gram)	1/5	4/5
precision (3-gram)	0/4	2/4
precision (4-gram)	0/3	1/3
brevity penalty	6/7	6/7
BLEU	0%	52%

Table 2.1: BLEU computation example

modified to

$$BLEU = BP. \exp\Big(\sum_{n=1}^{4} w_n \log p_n\Big) \tag{2.9}$$

where, BP is the brevity penalty and p_n are the modified precision counts, given by

$$p_n = \frac{\sum_{\mathcal{C}\in\{Candidates\}} \sum_{\text{n-gram}\in\mathcal{C}} Count_{clip}(\text{n-gram})}{\sum_{\mathcal{C}\in\{Candidates\}} \sum_{\text{n-gram}\in\mathcal{C}} Count(\text{n-gram})} \tag{2.10}$$

The brevity penalty BP penalizes translations that are smaller than the reference translation. The original paper sets BP to $e^{(1-\frac{r}{c})}$ if $c \leq r$ and sets BP to 1 if $c > r$, where c is the length of the candidate translation and r is the effective reference corpus length. The standard version of BLEU computation uses uniform weights $w_n = 1/4$.

For the WEATHERGOV dataset Liang *et al.* (2009) we also report a modified BLEU score cBLEU Angeli *et al.* (2010). cBLEU does not penalize numerical differences upto five units. For example, cBLEU will not penalize "minimum temperature of 60" when compared to the reference "minimum temperature of 63" (since $|60 - 63| < 5$)

2.6.2 NIST

The BLEU metric uses $n-$gram scoring to compute the quality of the candidate translations. The score uses a geometric mean of the $n-$gram occurrences across all $n(1-4$ for BLEU_4) and hence weights proportional differences in co-occurrences across n equally, which leads to variances due to low co-occurrences for larger n. We could take the arithmetic mean of $n-$gram counts to circumvent this issue. Additionally, it is better to weight the presence of more informative $n-$grams heavily compared to less informative $n-$grams (give more weight to $n-$grams that occur less frequently compared to more frequently occurring $n-$grams based on their information value). NIST

(Doddington (2002)) takes into account the above 2 observations and proposes a modification on top of the BLEU (Papineni *et al.* (2002)) metric.

The information content of an $n-$ gram $w_1, \ldots, w_n$ is computed as

$$Info(w_1, \ldots, w_n) = \log_2 \left(\frac{the\ \#\ of\ occurrences\ of\ w_1, \ldots, w_{n-1}}{the\ \#\ of\ occurrences\ of\ w_1, \ldots, w_n} \right) \tag{2.11}$$

Based in the information content of the $n-$grams the NIST score is computed as follows

$$NIST = \exp\left\{\beta \log^2 \left[\min\left(\frac{L_{sys}}{L_{ref}}, 1\right)\right]\right\}. \sum_{n=1}^{N} \left\{ \frac{\sum\limits_{all\ w_1,\ldots,w_n\ that\ co-occur} Info(w_1, \ldots, w_n)}{\sum\limits_{all\ w_1,\ldots,w_n\ in\ output} (1)} \right\}$$

The first term in the above equation corresponds to the brevity penalty and the second term computes the average of the $n-$gram co-occurrence scores. β is chosen to set the brevity penalty to 0.5 when the number of words in the output is $\frac{2}{3}$ of the average number of words in the reference sentence, L_{ref} is the average number of words in the reference sentences averaged over all reference examples and L_{sys} is the number of words in the translation being scored.

2.6.3 ROUGE

ROUGE(Lin (2004)) (**R**ecall-**O**riented **U**nderstudy for **G**isting **E**valuation) was proposed in the context of automatic evaluation of quality of machine generated summaries. BLEU metric is precision based and measures the quality of generated text by counting the match between $n-$grams in generated text with a set of reference gold standard text(s). ROUGE on the other hand is a recall based metric and is computed using

$$ROUGE_N = \frac{\sum\limits_{S \in \{Reference\ summaries\}} \sum\limits_{gram_n \in S} Count_{match}(gram_n)}{\sum\limits_{S \in \{Reference\ summaries\}} \sum\limits_{gram_n \in S} Count(gram_n)}$$

where n is the length of the $n-$gram, $Count_{match}(gram_n)$ is the number of $n-$grams co-occurring in a set of reference summaries and a candidate summary. The count in

the denominator increases if we have more reference summaries. Essentially, adding more gold standard candidates increases the number of alternate summaries and hence we can control for specific aspects of the "quality" of the generated text we want to measure. Other variants of the ROUGE metric have been proposed such as ROUGE_L, ROUGE_W and ROUGE_S. These are however focussed on evaluation of summarization systems and hence we do not consider these metrics for our evaluations. We evaluate our model generated text using BLEU_4, NIST_4 and ROUGE_4 and use one gold standard reference from wikipedia for each input infobox.

We now proceed to discuss the related work in the literature for the NLG from structured data problem. We also discuss 2 important datasets and provide a describe to two state of the art models which are used a baselines for comparsion with our work.

Chapter 3

Related Work

Natural Language Generation (NLG) is an active area of research. NLG problems have broad practical applicability, and researchers have developed multiple techniques for building NLG systems. Prior work suggests that this task can be divided into 3 phases *viz.*, Content planning, Sentence planning and Surface realization(Reiter and Dale (2000)). Existing approaches to address each of these 3 phases include (i) rule-based systems (ii) modular statistical approaches that use data-driven techniques for the three phases (content planning, sentence planning and surface realization) (iii) hybrid approaches relying on a combination of handcrafted rules and corpus statistics and (iv) more recent neural network based models.

In the next section, we give an overview of existing techniques and follow it up with a brief discussion on some prominent datasets and tasks for NLG from structured data. We conclude with a detailed discussion of two state of the art techniques *viz.*, Mei *et al.* (2016*b*) and Lebret *et al.* (2016) which are used as baseline models for comparison with our work.

3.1 Techniques

Traditional NLG systems treated the generation problem as a sequence of decisions. These decisions are influenced by the process humans use to generate text. They include (i) content planning - choosing the relevant parts of the input that need to be described (ii) sentence planning - deciding which previously selected portion of the input goes into the appropriate sentence in the generated output and (iii) surface realization - actual generation of text. The first step roughly corresponds to the question "What to say?" and the next two steps focus on answering the question "How to say it?". The rest of the section is organised as follows; We first start with a discussion of rule based systems,

follow it up with a discussion on statistical methods, hybrid systems and finally present neural network based papers.

Rule based systems

Early systems were built using hand-crafted specifications and rules. They focussed on domain specific problems like generating route descriptions, weather report generation *etc.* Dale *et al.* (2003) propose a system for generating route descriptions for navigational assistance. They used 2 corpora; the first corpus had 49 spoken route descriptions and the second corpus contained 30 written route descriptions. Their goal was to capture different modes of navigation and the means of communication used. They observe that humans have certain preferences in describing routes (for instance, humans tend to describe routes by landmarks and salient features rather than describe in terms of distances and travel times) and incorporate rules to capture these observations into their NLG system. Reiter *et al.* (2005) on the other hand focus on weather forecast generation and propose the SUMTIME-MOUSAM system to generate forecasts for offshore oil rigs in the North sea. Their corpus consisted of 2,123 instances (meteorological data and corresponding forecasts) and contained a total of 22.985 words. They analyzed the differences in the way how different humans chose to translate the data into words for the forecast. This analysis was then used to derive rules for choosing the right words to describe non-linguistic data (in this context, numerical weather data). In contrast, Green (2006) aims to generate arguments based on genetic counselling documents. They use a domain model to specify representative genetic diseases using a small set of categories (for example, test result, symptom, genotype). They build rules for generating descriptions based on an analysis of the corpus and the description of standard practices in genetic counselling. The domain model is used to derive qualitative constraints based on the relations between different categories and these constraints are used to back the arguments generated by the grammar. Galanis and Androutsopoulos (2007) present the NATURALOWL system for generating descriptions of instances or classes based on an ontology that is linguistically annotated (OWL DL). The system selects facts directly relevant to the instance and has provision to include facts further away in the ontology. The chosen facts are ordered using ordering annotations. The surface realization

phase is implemented via templates which consist of slots that are filled by the relevant properties from chosen facts. Turner *et al.* (2009) propose models for generating approximate descriptions from geo-referenced data using a 2 step process. In the first step, they use domain knowledge (meteorological knowledge) for selecting a frame of reference for description and then use constraints imposed by the end-user for selecting values within a frame of reference. They use the ROADSAFE corpus which consists of 431 examples and a total of 29, 857 words. It is important to note that the datasets used in the above systems were small and contained examples from a very restricted domain, enabling the use of rule-based systems. However, rule-based systems do not scale well for diverse domains and statistical methods are more appropriate in such contexts.

Statistical Data driven models

Statistical data-driven models for generation focussed on the 3 steps (content planning, sentence planning and surface realization) have been proposed in the literature. Duboue and McKeown (2002) build a system to automatically generate briefings of patients who have undergone cardiac surgery. A corpus of 24 briefings recorded over a 2 month period was used. Further they tagged phrases that conveyed content from 223 tags that were marked by physicians to indicate 39 important categories of information. The proposed content planner was represented as a tree and they use evolutionary algorithms to explore the space of possible trees and a fitness function to score sentences.

Ratnaparkhi (2002) focusses explicitly on surface realization and assume that the other 2 steps are performed using relevant systems. The paper targets a restricted domain of air travel and propose trainable NLG systems that map semantics to words and use attribute-value pairs as a semantic representation. The model finds the most probable word sequence with respect to a maximum entropy probability model. Additionally, they also propose a model that produce a highest probability word sequence that is consistent with a dependency-style grammar of phrase segments. A training set of 6000 examples and a test set of 1946 examples was used, and their annotations used 26 attributes to describe flights.

Barzilay and Lee (2004) on the other hand propose systems for 2 tasks; information ordering and extractive summarization. Domain specific content models (implemented

as Hidden Markov Models) were used to represent topics and shift within topics in the context of content planning. These content models are used to order a set of items (fields). The authors created a corpora from five different domains; earthquakes, clashes between armies and rebel groups, drug-related criminal offenses, financial reports and summaries of aviation accidents. Each domain contained around hundred examples, average sentence length of around ten words and vocabulary in the order of thousands of tokens.

Barzilay and Lapata (2005) also focus on content planning and propose techniques to automatically learn content selection rules from a corpus and its related database. Contextual dependencies are captured by treating content selection as a collective classification problem. The models are evaluated on the sports domain and experiment with a corpus of 468 texts with an average summary length of 46.8 words and their database contains 73,400 entries describing performance of players and their teams. Barzilay and Lapata (2006) focus on the problem of aggregation; where the task is to combine multiple linguistic structures into a single sentence. Sentence planning was formulated as a supervised set learning problem with each partition corresponding to a sentence. The dataset used was the same as the one used by Barzilay and Lapata (2005).

Belz (2008) propose a weather forecasting system using probabilistic generation-space models and use the same corpus used by Reiter *et al.* (2005). Essentially, the idea is that generation process is an incremental specification of the output and the representations at each step of the generation process form a generation space underlying the NLG system. A Probabilistic Context-free Representationally Underspecified (pCRU) generation was used for content planning and surface realization. Chen and Mooney (2008) propose an end-to-end system for generating sportscasts for simulated Robosoccer games. The model learns correspondences between human commentaries and game states(presented as tabular information). Experiments were conducted on the ROBOCUP dataset that contains 1,539 scenarios with a vocabulary of 214 words, each scenario contains an 2.4 records on average and a description of average length 5.7 words.

Liang *et al.* (2009) use a probabilistic generative model (based on hidden semi-Markov models) to choose important facts and then produce words coherent with the

chosen facts. Experiments were conducted on the WEATHERGOV dataset which has a vocabulary of 350 words and dataset of 22,146 examples. Angeli *et al.* (2010) model content selection and surface realization as a sequence of local decisions. Each decision is trained as a log-linear classifier and allow the decisions to depend on the entire past history to capture contexual dependencies for better generation. They experiment with the SUMTIME (Reiter *et al.* (2005)), ROBOCUP (Chen and Mooney (2008)) and WEATHERGOV(Liang *et al.* (2009)) datasets.

Lu and Ng (2011) propose end-to-end methods for sentence planning and surface realization and use a reduction-based weighted binary synchronous context-free grammar for generation from logical forms along with a generative model to establish phrasal correspondence between natural language word-sequences and logical sub-expressions. The GEOQUERY dataset consisting 880 queries on the US geography was used and was manually labelled with logical forms ($\lambda-$expressions). Konstas and Lapata (2012) on the other hand, use a probabilistic CFG to globally describe the structure in the input. The grammar is represented as a weighted hypergraph and the generation task is to find the best derivation tree based on the given input. Experiments were conducted on ROBOCUP, WEATHERGOV and ATIS (Air travel domain containing 5,426 scenarios) datasets.

The above mentioned papers target richer domains compared to the rule-based systems. However, these datasets and models use moderately sized datasets and are not diverse (each dataset belongs to a specific domain). There is an increasing interest in building systems that can be used to generate descriptions for examples belonging to diverse domains. Recent work has made attempts to build NLG systems for diverse domains using large scale datasets. We discuss hybrid approaches to building NLG systems next and then follow it with a discussion of neural network based approaches.

Hybrid models

There has been work exploring scenarios where it is useful to leverage a combination of hand-crafted rules and corpus statistics. Langkilde and Knight (1998) propose the Nitrogen generation system (for Machine Translation) which uses a combination of symbolic rules with linguistic information modelled based on statistical models learnt

using online text corpora. The central idea is to generate a set of candidate sentences and use a statistical model to rank the candidates and then output the highest scoring candidate. Soricut and Marcu (2006) propose the use of WIDL-expressions (Weighted Interleave, Disjunction and Lock) for machine translation and summarization. They note that systems such as Nitrogen (Langkilde and Knight (1998)) cannot integrate the properties of the input text and target text simultaneously. The paper suggests the use of WIDL-expressions as a representation formalism which lets them integrate input and target language preferences and tune the model for maximum performance. Mairesse and Walker (2011) observe that statistical methods for NLG can help optimize for grammaticality and naturalness of generated sentences using large training data, but these methods are unable to produce stylistic variations that is perceived by humans. A language generator (PERSONAGE) is described, which has parameters that are based on psychological findings pertaining to the linguistic reflexes of personality. A base generator is used for producing multiple candidates and human feedback is used to incorporate personality/style of each candidate. This is then used to train statistical models for predicting the style related parameters. In the recent past there has been a significant increase in the size of training datasets for machine learning models and these have been especially successful in the context of neural network based deep learning models which we discuss next.

Neural network based models

Over the past few years, neural NLG models based on the encode-attend-decode paradigm have been used successfully for tasks like machine translation (for example, Bahdanau *et al.* (2014)), summarization (for example, Rush *et al.* (2015), Chopra *et al.* (2016), Nema *et al.* (2017)), dialog (Yao *et al.* (2015)) *etc.* Lebret *et al.* (2016) propose a feed-forward neural network (Section 3.3) based conditional language model for NLG from structured data. On the other hand, we use a sequence model and suggest improvisations based on the peculiar characteristics of the biography generation task. Specifically we use neural components to exploit the hierarchy in the input data. Hierarchical encoders have been used in the past in the context of summarization (Nallapati *et al.* (2016)), document classification (Yang *et al.* (2016)), dialog systems (Serban *et al.* (2016)), *etc.*

Unlike our setup, these papers deal with unstructured data (sentences and words vs fields and values).

We now move to discussing 2 datasets (WEATHERGOV and WIKIBIO) and related tasks for NLG from structured data.

3.2 Datasets and Tasks

Natural Language Generation from structured data has been studied in the context of small datasets such as WEATHERGOV (Liang *et al.* (2009)), ROBOCUP (Chen and Mooney (2008)), NFL RECAPS (Barzilay and Lapata (2005)) and PRODIGY-METEO (Belz and Kow (2009)). Recently Mei *et al.* (2016*b*) proposed an RNN/LSTM based neural encoder-decoder models with attention for the WEATHERGOV and ROBOCUP datasets. These datasets contain very few examples (order of tens of thousands). Further, these datasets have a vocabulary of 350 words each. Our work focuses on generating biographies from structured input (Wikipedia infoboxes). The WEATHERGOV dataset task is closely related to the biography generation problem. For our experiments, we compare our models on the WIKIBIO Lebret *et al.* (2016) and WEATHERGOV datasets. We also experiment with two new datasets for biography generation in French and German. The details of these multilingual datasets are presented along with the details of the English dataset in Section 3.2.2.

3.2.1 WEATHERGOV

The WEATHERGOV dataset contains detailed description of local weather forecast and a corresponding short forecast report. The dataset was created by collecting local weather forecasts for $3,753$ US cities over a period of 3 days. Further, for each city, day pair, two scenarios were created corresponding to the day forecast and night forecast respectively. Figure 3.1 shows a sample scenario from the WEATHERGOV dataset. The corresponding description for the table from above is "a 20 percent chance of showers after midnight. increasing clouds, with a low around 48 southwest wind between 5 and 10 mph".

```
temperature(time=17-06, min=48, mean=53, max=61)
windSpeed(time=17-06, min=3, mean=6, max=11)
windDir(time=17-06, mode=SSW)
gust(time=17-06, min=0, mean=0, max=0)
skyCover(time=17-21, mode=0-25)
skyCover(time=02-06, mode=75-100)
precipChance(time=17-06, min=2, mean=14, max=20)
rainChance(time=17-06, mode=someChance)
```

Figure 3.1: Sample scenario from WEATHERGOV dataset

The input contains of hour-by-hour readings of temperature, sky cover, chance of rain etc., representing the state at that particular time window. The readings are aggregated over the time interval under consideration. The dataset contains $22,146$ scenarios and each scenario contains 36 records. The records are of 12 types and contain only integer and categorical fields. The average description in the dataset contains 28.7 words. Just to be clear, the task here is to generate a weather forecast based on the tabulated meteorological measurements.

3.2.2 WIKIBIO

The WIKIBIO dataset was introduced by Lebret *et al.* (2016). It has 728,321 biography articles taken from English Wikipedia. Each article has a corresponding infobox which serves as the structured data input to our models. The task is to generate the first sentence of the wikipedia article (a one line description of the person). A sample infobox is shown in Figure 1.2 along with a corresponding target description.

Description - "V. Balakrishnan (born 1943 as Venkataraman Balakrishnan) is an Indian theoretical physicist who has worked in a number of fields of areas, including particle physics, many-body theory, the mechanical behavior of solids, dynamical systems, stochastic processes, and quantum dynamics."

In figure 1.2, we refer to "Born", "Residence", "Nationality" etc., as fields and "1943", "India", "Indian" etc., as values.

We use the original data splits made available by Lebret *et al.* (2016) [1] (they randomly split the dataset into train (80%), valid (10%) and test (10%)) . We were also interested in testing our models on different languages. To this end, we introduce two

[1] https://github.com/DavidGrangier/wikipedia-biography-dataset

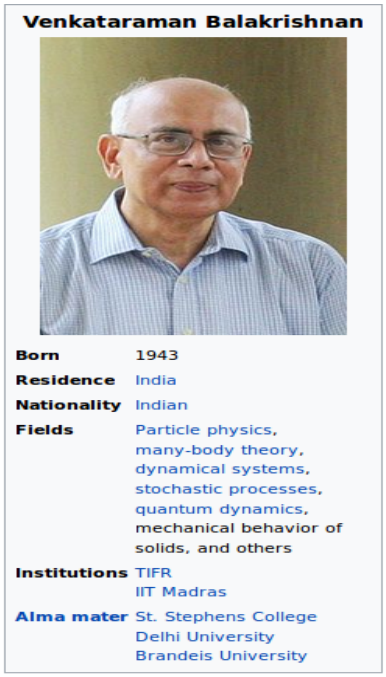

Figure 3.2: Infobox for Prof. V. Balakrishnan

new datasets for biographies in French and German. These were created using a similar procedure as used for the English WIKIBIO dataset. These datasets were also randomly split into train (80%), valid (10%) and test (10%) splits. The statistics from all the 3 datasets are presented in Table 3.1.

Language	# examples	Vocab Size	Average Sentence Length
EN	728321	426910	26.05
DE	54985	143108	32.29
FR	177674	297130	36.49

Table 3.1: WIKIBIO Dataset statistics

Our current models focus on generating descriptions in only one language. In the future, these datasets can be used to train models to generate descriptions from structured data in multiple languages.

3.3 Baseline approaches

We now discuss two baseline approaches to NLG from structured data. The first paper (Mei *et al.* (2016*b*)) proposes a model to perform the joint task of content selection and surface realization using an encoder-aligner-decoder model. The work in Mei *et al.*

(2016*b*) was in the context of WEATHERGOV dataset. We adapt their model and compare its performance with our models. The second paper (Lebret *et al.* (2016)) proposes a conditional feedforward neural network model for generating natural language descriptions from structured data. We use the WIKIBIO dataset introduced in this paper, for evaluating our models.

3.3.1 Baseline 1: Coarse-to-Fine alignment model

The input is given as a pair of scenarios $(r^{(i)}, x^{(i)})$, where $r^{(i)}$ is the set of records (events occuring in a game/measurements of a certain attribute - for example, temperature) and $x^{(i)}$ is the corresponding natural language description (commentary/forecast). The model consists of 3 main components (i) Encoder (ii) Coarse-to-fine aligner and (iii) Decoder. We present an overview of each of the components.[2]

The sentence generation is cast as an inference over the model $P(x_{1:T}|r_{1:N})$ [3], where $r_{1:N} = (r_1, r_2, \ldots, r_N)$ is the set of input records and $x_{1:T} = (x_1, x_2, \ldots, x_T)$ is the target description.

Figure 3.3 depicts the model architecture[4].

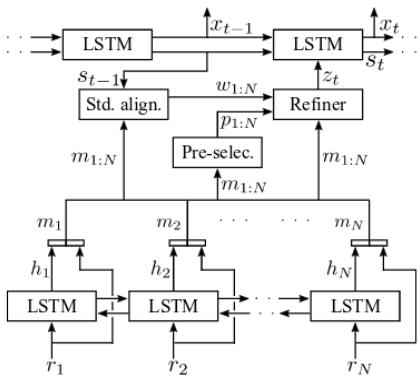

Figure 3.3: Coarse-to-fine grain alignment model architecture

Encoder

Mei *et al.* (2016*b*) use a bidirectional LSTM encoder to encode the input sequence

[2]Please refer to section 2.5 for relevant background in encoder-decoder models
[3]Notation borrowed from Mei *et al.* (2016*b*)
[4]Model architecture taken from Mei *et al.* (2016*b*)

$r_{1:N} = (r_1, r_2, \ldots, r_N)$ and return the hidden state representations $h_{1:N} = (h_1, h_2, \ldots, h_N)$ (each state h_j is a concatenation of states from forward and backward LSTM).

Coarse-to-fine aligner

The input records are over-determined and only a small number of the records are relevant to the output description. The authors state that standard alignment mechanisms (Bahdanau *et al.* (2014)), have a limitation on the accuracy because they need to scan over the entire set of input records. The model chooses the relevant content for generating the output at each step t using a two-step process. They propose a coarse-to-fine grained aligner that first decides important records (pre-selector) and then refine (refiner) the attention on the important parts of the input.

The attention weights are computed based on a combination of multiple representations of the input (the model uses a concatenation $m_j = [r_j; h_j]$ of the original record r_j and the corresponding hidden state h_j). It has been observed that multiple representations lead to better alignments than using just the hidden states (Mei *et al.* (2016*a*)). The pre-selector computes the probability p_j for a record r_j being selected. The standard attention weights w_{tj} (Bahdanau *et al.* (2014)) are computed separately. The refiner combines the pre-selector weights and standard alignments to arrive at a refined attention weight, based on the following computations

$$
\begin{aligned}
p_j &= \textit{sigmoid}(q^\top \textit{tanh}(Pm_j)) \\
\beta_{tj} &= v^\top \textit{tanh}(Ws_{t-1} + Um_j) \\
w_{tj} &= \exp(\beta_{tj}) / \sum_k \beta_{tk} \\
\alpha_{tj} &= \frac{p_j w_{tj}}{\sum_k p_k w_{tk}} \\
z_t &= \sum_j \alpha_{tj} m_j
\end{aligned}
$$

where z_t is the new refined context that is fed into the decoder LSTM and q, P, v, W, U are model parameters.

The pre-selector p_j assigns large probabilities to a small set of relevant records from the input and low probabilities to the rest of the records. This pre-select scores then refine the standard attention weights and hence allow the model to focus only on the most relevant records from the input (the input records are over-determined and only a few a relevant for the generation task).

Decoder

The decoder is a standard LSTM decoder which takes in the last output word x_{t-1}, the previous decoder state s_{t-1} and the current context vector z_t to predict the next word in the sequence x_t by producing the conditional distribution on the next word $P(x_t|x_{1:t-1}, r_{1:N})$.

Regularization

The pre-selector score p_j, specifies the degree to which the j-th record should be selected. The sum $\sum_{j=1}^{N} p_j$ serves as a proxy for the total number of pre-selected records (γ). To ensure that a set number of records be given high scores, they introduce a regularization term $(\sum_{j=1}^{N} p_j - \gamma)^2$. γ is set based on performance on the validation dataset. Additionally, they introduce the term $1-\max(p_j)$ to ensure that atleast one record needs to be pre-selected.

The loss function is given by,

$$
\begin{aligned}
L &= -\log P(x^*_{1:T}|r_{1:N}) + \Big(\sum_{j=1}^{N} p_j - \gamma\Big)^2 + (1 - \max(p_j)) \\
&= -\sum_{t=1}^{T} \log P(x^*_t|x_{0:t-1}, r_{1:N}) + \Big(\sum_{j=1}^{N} p_j - \gamma\Big)^2 + (1 - \max(p_j))
\end{aligned}
$$

where $x^*_{1:T}$ is the true description, x_0 is a special start symbol, $r_{1:N}$ are the input records, p_j's are pre-selector scores and γ is a regularization parameter.

They tune the hyper-parameters based on BLEU score on the development dataset, and set the LSTM size to be 500 and $\gamma = 8.5$. Further, they use the Adam optimizer with a mini-batch of 100 examples for training. The best model is selected based on the

BLEU score on the validation set.

3.3.2 Baseline 2: Table conditioned Neural Language Model

The infobox is given as a set of field/value pairs and the language model proposed in Lebret *et al.* (2016) is conditioned on these pairs. The three main components of the model are (i) local conditioning (ii) global conditioning and (iii) copy actions. We start with a description of some notation and then describe the model components in the subsequent sections.

Notation

We now present some notation from the Lebret *et al.* (2016) paper. Boldface upper case letters ($\mathbf{X}, \mathbf{W}$ etc.) are used to denote matrices (parameter matrices and embedding matrices) and lower case letters are used to denote vectors (for example, $\mathbf{b}$). The concatenation of two vectors $\mathbf{v_1}$ and $\mathbf{v_2}$ is denoted by $[\mathbf{v_1}; \mathbf{v_2}]$.

Embedding layer

Neural models use high-dimensional vectors to represent atomic symbols such as words and other tokens of interest. The vectors are referred to as embeddings and capture meaningful semantic properties of words. Popular techniques for word embeddings include Word2Vec (Mikolov *et al.* (2013)) and GloVe (Pennington *et al.* (2014)). Lebret *et al.* (2016) use embeddings derived using Hellinger PCA (Lebret and Collobert (2014)).

An embedding layer is an abstraction that takes a matrix of embeddings and maps words/symbols to their vector representations. The embedding lookup from an embedding matrix $\mathbf{X} \in \mathbb{R}^{N \times d}$ is given by

$$\psi_{\mathbf{X}}(x_i) = \mathbf{X}_i \in \mathbb{R}^d$$

here $\mathbf{X_i}$ corresponds to the embedding of x_i at row i of the 2-d embedding matrix($\mathbf{X}$).

$$\psi_{\mathbf{X}}(x_i, x_j) = \mathbf{X}_{i,j} \in \mathbb{R}^d \tag{3.1}$$

when $\mathbf{X}$ is a 3-d embedding matrix and $\mathbf{X}_{i,j}$ is the embedding at the index (i, j).

The embedding lookup on a sequence $x_1, \ldots, x_T$ is given by

$$\psi_{\mathbf{X}}(s) = [\psi_{\mathbf{X}}(x_1); \ldots; \psi_{\mathbf{X}}(x_T)] \in \mathbb{R}^{Td}$$

Linear layer

Linear layer applies a linear transformation on a given input $x \in \mathbb{R}^n$

$$\gamma_\theta(x) = \mathbf{W}\mathbf{x} + \mathbf{b} \in \mathbb{R}^m \tag{3.2}$$

here $\mathbf{W} \in \mathbb{R}^{m \times n}$ and $b \in \mathbb{R}^m$ are the weight matrix and bias term respectively. $\theta = \{\mathbf{W}, \mathbf{b}\}$ is used to denote the set of trainable parameters.

We this background in place, we now proceed to discuss the details of the model proposed by Lebret *et al.* (2016).

Context conditioning

Given a description $s = w_1, \ldots, w_T$ an n-gram language model estimates

$$\begin{aligned} P(s) &= P(w_1, w_2, \ldots, w_T) \\ &= \prod_{t=1}^{T} P(w_t | w_{t-1}, \ldots, w_2, w_1) \\ &\approx \prod_{t=1}^{T} P(w_t | w_{t-1}, \ldots, w_{t-(n-1)}) \\ &= \prod_{t=1}^{T} P(w_t | c_t) \end{aligned}$$

Here c_t is referred to as the n-gram context of the word w_t. The n-gram context c_t

is represented by the concatenation of embeddings of words appearing in the context.

$$\psi_{\mathbf{E}}(c_t) = [\psi_{\mathbf{E}}(w_{t-(n-1)}); \ldots ; \psi_{\mathbf{E}}(w_{t-1})]$$

where $\mathbf{E} \in \mathbb{R}^{|\mathcal{W}| \times d}$ is the word embedding matrix and $|\mathcal{W}|$ is the size of the shortlist vocabulary. In addition to conditioning on c_t (which is the standard n-gram language model), Lebret *et al.* (2016) also condition the probability distribution $\mathbf{P}$ on the contents of the table. Specifically, they use local and global information from the infobox for conditioning as described below.

Local conditioning

Local conditioning aims to capture the information in the table that has already been described. This helps the model keep track of what parts of the table have already been described and parts that remain to be described. Every word in the table can be described by the field that is belongs to and its relative position within the table. Usually the words within the table are listed in their order of importance and we would like to exploit this information. Further, words towards the end of the field acts as signal to the model indicating that it needs to describe contents from a different field. Specifically, consider a word w in the description. Let $\mathcal{F}$ denote the set of all possible fields. Any word w from the target description can be described using the following set

$$z_w = \{(f_i, p_i^+, p_i^-)\}_{i=1}^m$$

where m is the number of times the word w occurs in the table and f_i's are the fields in which the word occurs. p_i^+ and p_i^- indicate the positions from start and end for the word in field $f_i \in \mathcal{F}$ from the table respectively. Consider the following sample infobox for illustration

In the infobox presented in figure 3.4,[5]

$$z_{linguistics} = \{(fields, 8, 4); (known\ for, 4, 13)\} \tag{3.3}$$

[5]Illustration borrowed from Lebret *et al.* (2016)

Frederick Parker-Rhodes	
Born	21 November 1914 Newington, Yorkshire
Died	2 March 1987 (aged 72)
Residence	UK
Nationality	British
Fields	Mycology, Plant Pathology, Mathematics, Linguistics, Computer Science
Known for	Contributions to computational linguistics, combinatorial physics, bit-string physics, plant pathology, and mycology
Author abbrev. (botany)	Park.-Rhodes

Figure 3.4: Infobox for illustrating local conditioning

Each field f_i is associated with two separate embeddings, corresponding to starting position (f_i, p_i^+) (embedding matrix $\mathbf{Z}^+$) and position from end (f_i, p_i^-) (embedding matrix $\mathbf{Z}^-$) and we refer to the combined matrix as $\mathbf{Z} = \{\mathbf{Z}^+, \mathbf{Z}^-\}$. Note that both $\mathbf{Z}^+$ and $\mathbf{Z}^-$ are 3-d embedding matrices.

Every word w in the $n-$gram context (c_t) can be described using an embedding z_w as specified above. The local conditioning variable z_{c_t} on the context c_t is then defined as the concatenation of embeddings from individual words given by the max-aggregation as follows

$$\psi_{\mathbf{Z}}(z_w) = [\max\{\psi_{\mathbf{Z}^+}(f_i, p_i^+), \forall(f_j, p_i^+, p_i^-) \in z_w\}; \max\{\psi_{\mathbf{Z}^-}(f_i, p_i^-), \forall(f_j, p_i^+, p_i^-) \in z_w\}] \quad (3.4)$$

here $\psi_{\mathbf{Z}^+}(f_i, p_i^+)$ and $\psi_{\mathbf{Z}^-}(f_i, p_i^-)$ are embedding lookups from 3-d embedding matrices (equation (3.1)). Figure 3.5 demonstrates a sample scenario for computation of equation (3.4) for the example given in equation (3.3).

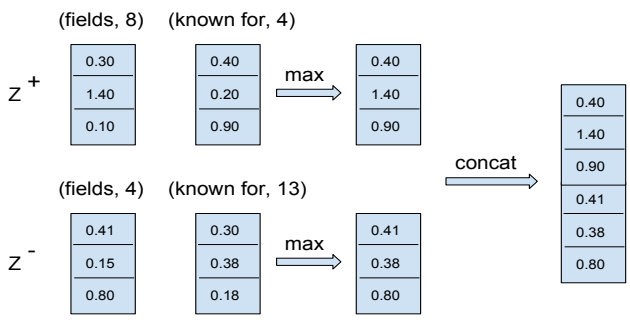

Figure 3.5: Local conditioning illustration

Global conditioning

Global conditioning captures the "global" information contained in the table and is specified via two seperate signals; fields in the table and the words appearing in the

table. The actual set of fields that appear in the infobox help in deciding the structure for generation. For instance, the fields that appear in an infobox for a sportsperson will be very different from the fields corresponding to a politician. This information is captured using the global field conditioning g_f. For conditioning over fields, we define $\mathcal{F}^q \in \mathcal{F}$ to be the set of all fields in the table q, and obtain the representation for field conditioning from embeddings $\mathbf{G}^f$, using

$$\psi_{\mathbf{G}^f}(g_f) = \max\{\psi_{\mathbf{G}^f}(f_j), \forall f_j \in \mathcal{F}^q\} \tag{3.5}$$

Further, the actual words appearing in the table provide information complementary to the fields. For example, both a basketball player and hockey player might have similar fields but the specific words appearing in those fields can help distinguish one from another. The global word conditioning variable g_w captures information contained in the words appearing in the infobox. For global conditioning over words, we define $\mathcal{Q}$ as the set of all words in the table q, and obtain the representation using embeddings $\mathbf{G}^w$ as follows,

$$\psi_{\mathbf{G}^w}(g_w) = \max\{\psi_{\mathbf{G}^w}(w), \forall w \in \mathcal{Q}\} \tag{3.6}$$

Note that the component-wise max aggregation in equation (3.5) and (3.6) are implemented similar to the example illustrated in figure 3.5.

The context input to the model $x = \{c_t, z_{c_t}, g_f, g_w\}$ is represented using a latent representation $h(x)$ by performing a non-linear transformation

$$h(x) = \textit{tanh}(\gamma_{\alpha_1}(\psi_{\alpha_2}(x))) \tag{3.7}$$

where $\alpha_1 = \{\mathbf{W_1}, \mathbf{b_1}\}$ are the weights and biases for the linear transformation (equation (3.2)) and $\alpha_2 = \{\mathbf{E}, \mathbf{Z}^+, \mathbf{Z}^-, \mathbf{G}^f, \mathbf{G}^w\}$ are embedding matrices.

Copy actions

The biography descriptions usually contain words that are copied directly from the input infobox. For example, the shortlist vocabulary $\mathcal{W}$ might not necessarily contain all words for the description, but the model can copy words from the input infobox to

generate coherent descriptions. Lebret *et al.* (2016) allow for copying words $w \in \mathcal{Q}$ from the input infobox by scoring words in the combined vocabulary $\mathcal{W} \cup \mathcal{Q}$.

In-vocabulary words are given a score based on the latent context representation $h(x)$ (3.7) using

$$\phi^{\mathcal{W}}(x) = \gamma_{\alpha_3}(h(x)) \in \mathbb{R}^{|\mathcal{W}|}$$

where $\alpha_3 = \{W_2, b_2\}$ are the weight and bias parameters.

Each word appearing in the infobox is given an additional representation based on its occurrence in the table z_w using an additional embedding matrix $\mathbf{F}$, an intermediate representation $q(w)$ is computed and an additional score $\phi^{\mathcal{Q}}(x, w)$ is computed as specified by the following equations

$$q(w) = \max\{\textit{tanh}(\gamma(\psi_{\mathbf{F}}(f_j, p_i)), \forall (f_j, p_i) \in z_w\}$$
$$\phi^{\mathcal{Q}}(x, w) = h(x)^\top q(w)$$

The combined scores for word $w \in \mathcal{W} \cup \mathcal{Q}$ is given by,

$$\phi(x, w) = \phi^{\mathcal{W}}(x, w) + \phi^{\mathcal{Q}}(x, w)$$

These scores are passed through a softmax layer to obtain a distribution over the combined vocabulary $\mathcal{W} \cup \mathcal{Q}$.

Training

The loss function for a given training sentence $s = w_1, \ldots, w_T$ is given by

$$L(s) = -\sum_{t=1}^{T} \log P(w_t | c_t, z_{c_t}, g_f, g_w)$$

where the conditioning variables c_t, z_{c_t}, g_f, g_w are computed as described above. The models is trained using SGD with a learning rate of 0.0025.

3.4 Summary

In this chapter, we saw a brief overview of techniques from the literature for addressing the problem of generating Natural Language descriptions from structured data. We then discussed 2 important datasets and related NLG tasks and proceeded to discuss two baseline approaches. In the next chapter, we describe our proposed models for NLG from structured data and present the results from our experiments.

Chapter 4

Proposed Models for Biography Generation

4.1 Overview

The WIKIBIO dataset [1] (Lebret *et al.* (2016)), has around $700K$ infobox, biography pairs. These biographies belong to people from different walks of life (artists, athletes, politicians, etc.). Given these variations, it is hard to design rule based templates for biography generation. Recently neural models have been successfully applied to a wide variety of Natural Language Generation tasks (Sutskever *et al.* (2014), Bahdanau *et al.* (2014), Rush *et al.* (2015), Chopra *et al.* (2016), Nema *et al.* (2017)). Lebret *et al.* (2016) use a data-driven conditional language model implemented as a feed-forward network for the biography generation task. In contrast, we model the task based on the encode-attend-decode framework also known as the seq2seq model. We propose two models for the biography generation task. Our first model, the flat model builds on top of the seq2seq model (Sutskever *et al.* (2014), Bahdanau *et al.* (2014)). We then make a few observations on the properties of the structured data under consideration, leading us to our second model, the hierarchical model.

We experiment with the English WIKIBIO dataset and the WEATHERGOV dataset. Further, we introduce two new datasets for biographies in French and German. These were created using a similar procedure as used for the English WIKIBIO dataset. The details of these datasets were provided in Section 3.2.2.

The rest of the chapter is organized as follows; we start with setting up notation for discussing our proposed models. We then describe the flat model and the hierarchical model for biography generation. For each model we describe the rationale for our modelling choices. Finally, we describe the experimental setup and conclude with a discussion of results on the biography generation task.

[1] https://github.com/DavidGrangier/wikipedia-biography-dataset

4.2 Notation

The input infobox is given as a set of field, value pairs, $\mathcal{I} = \{(g_i, k_i)\}_{i=1}^{M}$, where g_i corresponds to field names, k_i is the list of words in the corresponding field and M is the total number of fields in the infobox. For instance in the sample infobox given in Table 4.1, $(g = occupation, k = professor, researcher)$ could be one such pair in the infobox. We refer to $occupation$ as the field and $(professor, researcher)$ as values.

Given the input infobox, the task is to generate a description $y = y_1, y_2, \ldots, y_m$, which is a sequence of m words.

Formally, our goal is to find the optimal sequence y^*, which maximizes the conditional probability $p(y|\mathcal{I})$ (probability of sequence given the infobox)

$$y^* = \underset{y}{\operatorname{argmax}} \prod_{y=1}^{m} p(y_t|y_1, \ldots, y_{t-1}, \mathcal{I}) \tag{4.1}$$

The biography generation task, requires us to model the conditional probability distribution on the right side of equation 4.1. We encode our assumptions on the properties of the data using this conditional probability distribution. Specifically, the conditional distribution is modelled using a neural encoder decoder framework with attention mechanism.

4.3 Flat Model

4.3.1 Model Description

The input infobox has a certain ordering of fields and values contained in it. The question that we consider is: “Can we capture the input infobox as a sequence?”. In other words, can the infobox be represented as an appropriate sequence? Let us consider a sample infobox presented in Table 4.1 with the corresponding description "John Doe (born 18 April, 1952) is a Cambridge educated British politician."

The one line description typically starts with the name and goes on to describe other

name	John Doe
birthdate	18 April 1952
birthplace	Oxford UK
occupation	senator, mayor
institution	Cambridge University
spouse	Jane Doe
children	Johnnie Doe

Table 4.1: Sample infobox

attributes of the person. Further note that there are certain words in the description (British, politician) that are based on words in the input infobox (senator, mayor, Oxford UK) but are not explicity present in the input infobox. Given the diversity of the domains to which the input examples belong to (sports, politics, arts *etc.*), it is very hard to design rule-based systems. Because of these characteristics, statistical models are well suited compared to rule-based systems.

We *flatten* the input infobox into a sequence as follows

"**name** John Doe **birthdate** 18 April 1952 **birthplace** Oxford UK **occupation** senator mayor **institution** Cambridge University **spouse** Jane Doe **children** Johnnie Doe"

The words in bold refer to the field names and are included in our flattened sequence to provide additional signals to our model about boundaries between different fields in the infobox. The representation of the infobox as the sequence above, lets us leverage seq2seq models (Sutskever *et al.* (2014)). The input sequence is encoded using an input encoder (RNN/LSTM/GRU). We have an attention mechanism (Bahdanau *et al.* (2014)) over the input tokens and an output decoder to generate the target description one word at a time.

Encoder

The input sequence $x_1, x_2, \ldots, x_T$ is mapped onto a sequence of hidden representations ($h_j, \forall j \in \{1, \ldots T\}$) using a bidirectional GRU.

$$h_j = GRU(h_{j-1}, e(x_j))$$

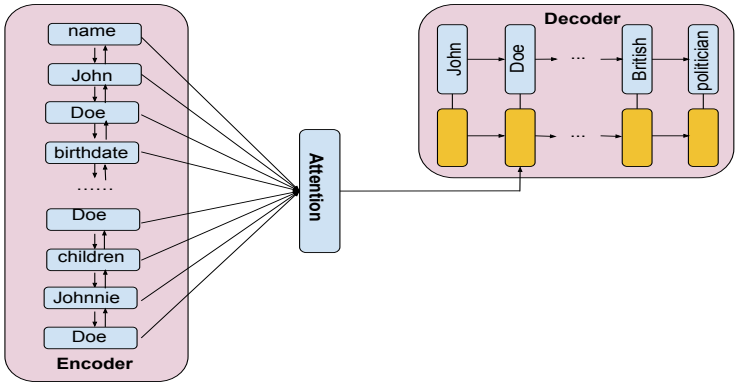

Figure 4.1: Flat model block diagram

where $e(x_j)$ is the d-dimensional embedding for word x_j and $h_j \in \mathbb{R}^{l_1}$ is the representation obtained using the bidirectional GRU. We use 300 dimensional pre-trained Glove (Pennington *et al.* (2014)) embeddings for our experiments. We have an additional unknown word symbol ("<unk>" token) to represent all words that do not appear in the shortlist vocabulary.

Attention based decoder

We now describe a GRU based decoder that produces the biography description one word at a time by appropriately paying attention to the relevant parts of the input. The attention over the input tokens is computed as follows

$$\begin{aligned} a_{t,j} &= v_w^T tanh(U_w s_t + V_w h_j) \\ \alpha_{t,j} &= \frac{\exp(a_{t,j})}{\sum_{l=1}^{T} \exp(a_{t,l})} \\ c_t &= \sum_{j=1}^{T} \alpha_{t,j} h_j \end{aligned} \tag{4.2}$$

where s_t is the current state of the decoder at time step t . $U_w \in \mathbb{R}^{l_1 \times l_2}$, $V_w \in \mathbb{R}^{l_1 \times l_1}$ and $v_w \in \mathbb{R}^{l_1}$ are parameters, l_2 is the size of the decoder's hidden state, T is the total number of words in the flattened input, c_t is the attention weighted context vector at

time step t.

Finally, the decoder state at each timestep is calculated using the following:

$$s_t = \text{GRU}_{dec}(s_{t-1}, [e(y_{t-1}); c_{t-1}]) \tag{4.3}$$

where $e(y_{t-1})$ is the embedding of the word generated at timestep $t-1$ and $[a; b]$ denotes the concatenation of vectors a and b. The decoder then computes a probability distribution over the vocabulary as follows:

$$y_t = \text{softmax}(W_o f(U_{dec} s_t + L_{dec} c_t)) \tag{4.4}$$

where $W_o \in \mathbb{R}^{|N| \times l_2}$, $U_{dec} \in \mathbb{R}^{l_2 \times l_2}$, $L_{dec} \in \mathbb{R}^{l_2 \times l_1}$ are parameters and $|N|$ is the size of the target vocabulary. Equation (4.4) models the probability distribution mentioned in equation (4.1) (conditional probability of the sequence given infobox and tokens generated so far) and is used to generate tokens for the target sentence. We chose f to be the identity function.

Copy Mechanism

The dataset has a vocabulary of around $426K$ for English, $143K$ for German and $297K$ for French. We do not consider all these words in our shortlist vocabulary but only choose the top $20K$ frequently occurring words as our shortlist vocabulary. This is similar to the setup of Lebret *et al.* (2016) and was primarily set to ensure that the softmax computation is tractable. Fortunately, a lot of words are copied directly from the input infobox onto the target description. We enable our model to copy the words from the input as a post-process step. This process is similar to the one presented in Luong *et al.* (2015). Specifically, we look at those time-steps(t) where the model predicts the unknown token (<unk>) and copy that input token which had the highest attention weight for the corresponding time-step. In other words, for the time-steps t where the model predicts the "<unk>" token we replace the "<unk>" token with the

input token $x_i, i \in \{1, \ldots, T\}$, where

$$i = \underset{j}{\operatorname{argmax}}(\alpha_{t,j})$$

where $\alpha_{t,j}$ is computed using equation (4.2).

We now proceed to discuss our next model, the hierarchy model.

4.4 Hierarchy Model

4.4.1 Model Description

In our first model we represented the infobox as a sequence and leveraged seq2seq models to address the biography generation task. Alternately, we notice that the infobox can be viewed as composed of fields which are themselves composed of corresponding values. In other words, the infobox can be thought of as a two level hierarchy. Our next model, explicitly encodes the hierarchy in the input infobox. Specifically we have two separate encoders (i) field encoder (level 1) and the (ii) infobox encoder (level 2). We now describe the field level and infobox level encoders.

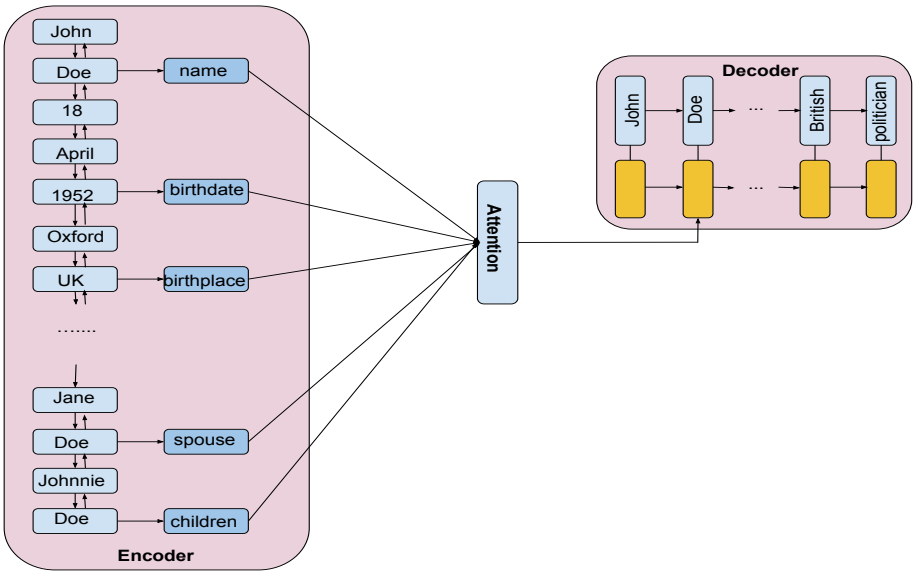

Figure 4.2: Hierarchy model block diagram

Field encoder

The field encoder computes a representation for each field in the infobox based on all values contained within the field. Let us consider a field g with its corresponding values $k = (w_1, w_2, \ldots, w_p)$. The representation of the field is computed by treating the values $w_1, w_2, \ldots w_p$ as a sequence. Note that these values do not form a natural language sentence in English (or German, French) but there is an order in which the values are presented (for example, occupations are often listed in the order of relevance to the person). We make a modelling choice to treat this as a sequence. Specifically, we use a GRU cell, which takes as input a d-dimensional embedding of the j-th value in the field at the j-th timestep.

$$h_j^w = \text{GRU}_f(h_{j-1}^w, e(w_j))$$

where $h_j^w \in \mathbb{R}^{l_1}$ is the hidden state vector of the GRU, $e(w_j)$ is the embedding for value w_j in the field. The final state of the GRU, h_p^w at timestep p can be treated as the representation for the field. Note that alternately we could have computed the field representation as a sum of the embeddings of all the values in the field. In practice, we found that using a GRU works better than simply summing the value representations.

Infobox encoder

This encoder computes a representation for the infobox based on all the fields in it. Consider an infobox $\mathcal{I}$ with fields $g_1, g_2, ..., g_n$. The representation of the infobox can be computed by treating the fields $g_1, g_2, ..., g_n$ as a sequence. The representation for each field is obtained by concatenating the embedding of the field name (name, birth-date, occupation, etc) with the field representation computed using the field encoder described above. We note that similar to the case of values within a field, the fields themselves have an inherent ordering. Fields such as name, date of birth, place of birth and important attributes of the person are usually presented in order. Hence we compute the infobox representation using a GRU cell which takes as input the concatenated field

representation of the i-th field at the i-th timestep.

$$h_i^g = \text{GRU}_q(h_{i-1}^g, [e(g_i); h_{p_i}^w]) \tag{4.5}$$

where $h_i^g \in \mathbb{R}^{l_2}$ is the hidden state vector of the GRU at timestep i, $e(g_i)$ is the embedding for the field name g_i and $h_{p_i}^w \in \mathbb{R}^{l_1}$ is the last state of the corresponding level 1 field encoder.

Attention based decoder

We now describe the GRU based decoder which produces one word at each time step by appropriately paying attention to values and fields. The attention over the values in any field is given by the following set of equations:

$$a_{t,j}^w = v_w^T tanh(U_w s_t + V_w h_j^w) \tag{4.6}$$

$$\alpha_{t,j}^w = \frac{\exp(a_{t,j}^w)}{\sum_{l=1}^{W} \exp(a_{t,l}^w)} \tag{4.7}$$

$$c_t^w = \sum_{j=1}^{W} \alpha_{t,j} h_j^w \tag{4.8}$$

where s_t is the current state of the decoder at time step t . $U_w \in \mathbb{R}^{l_1 \times l_3}$, $V_w \in \mathbb{R}^{l_1 \times l_1}$ and $v_w \in \mathbb{R}^{l_1}$ are parameters, l_3 is the size of the decoder's hidden state, W is the total number of values across all fields, c_t^w is the attention weighted value level context vector at time step t.

Similarly, the attention over fields is computed using the following set of equations:

$$b_{t,i}^g = v_g^T tanh(U_g s_t + V_g h_i^g)$$

$$\beta_{t,i}^g = \frac{\exp(b_{t,i}^g)}{\sum_{l=1}^{G} \exp(b_{t,l}^g)}$$

$$c_t^g = \sum_{i=1}^{G} \beta_{t,i} h_i^g \tag{4.9}$$

$U_g \in \mathbb{R}^{l_2 \times l_3}$, $V_g \in \mathbb{R}^{l_2 \times l_2}$ and $v_g \in \mathbb{R}^{l_2}$ are parameters, G is the total number of fields in the input, c_t^g is the field level context vector at time step t.

The value level attention weights are then reweighed based on the corresponding field level attention weights. The motivation behind combining the attention weights in the following fashion is that the weight given to the fields should have an influence on the values belonging to the particular field. The attention weights at both the levels are then combined using the following equations:

$$\alpha'_{t,j} = \frac{\alpha_{t,j}\beta_{t,F(j)}}{\sum_{l=1}^{W} \alpha_{t,l}\beta_{t,F(l)}} \tag{4.10}$$

$$c_t^{w'} = \sum_{j=1}^{W} \alpha'_{t,j} h_j^w \tag{4.11}$$

where $F(j)$ returns the field corresponding to the j-th value in the input, $c_t^{w'}$ is the new value level context vector.

Finally, the decoder state at each timestep is calculated using the following :

$$s_t = \text{GRU}_{dec}(s_{t-1}, [e(y_{t-1}); c_{t-1}^{w'}; c_{t-1}^{g}]) \tag{4.12}$$

where $e(y_{t-1})$ is the embedding of the word produced at timestep $t-1$. The decoder then computes a probability distribution over the vocabulary as follows:

$$y_t = \text{softmax}(W_o f(U_{dec} s_t + L_{dec} c_t^{w'} + M_{dec} c_t^{g})) \tag{4.13}$$

where $W_o \in \mathbb{R}^{|N| \times l_3}$, $U_{dec} \in \mathbb{R}^{l_3 \times l_3}$, $L_{dec} \in \mathbb{R}^{l_3 \times l_1}$ and $M_{dec} \in \mathbb{R}^{l_3 \times l_2}$ are parameters and $|N|$ is the size of the vocabulary. Equation (4.13) models the probability distribution of tokens given a context as specified in equation (4.1) (conditional probability of the sequence given infobox and tokens generated so far) and is used to generate the target description. We chose f to be the identity function.

Copy Mechanism

We enable our hierarchical model with copy mechanism. This is similar to the one presented in 4.3.1. Here we use the re-weighted attention weights (equation (4.10)) for choosing the appropriate word to be copied.

This concludes our discussion of models for biography generation. We now discuss some implementation details and then proceed to discuss our experimental setup and results on the generation task.

4.5 Implementation details

We implemented our models using TensorFlow, a library for performing numerical computations with inbuilt support for machine learning and deep learning. In particular, the library provides primitives for describing deep networks and computations on the networks (training and inference) and also provides support to run computations on GPUs. TensorFlow programs specify the network using a computation graph abstraction. The nodes in a computation graph indicate variables (vectors, scalars, tensors *etc.*). The graph also allows us to define operations that compute simple functions of one or more variables. This graph abstraction makes it easy to perform automatic differentiation. A TensorFlow program describes the computation graph for the underlying model along with details of procedures for training and performing inference on the networks.

Our implementation consists of 3 main components *viz.,* (i) Dataset preprocessing (ii) Neural network model and (iii) Postprocessing steps (for copy mechanism). We describe the individual components in the subsequent sections.

4.5.1 Dataset preprocessing

For the Flat model, we first generate the flat input infobox representation as mentioned in Section 4.3.1. To account for variable length inputs, we append flattened infoboxes with "<pad>" symbols. For the hierarchy model we perform additional processing to generate inputs for the RNNs in the encoder. For the English dataset, we retain the first 20 fields ($fields_per_box$) within any given infobox, and the first 5 values ($tokens_per_field$) within a given field. The fields and values are padded with "<pad>" symbols to ensure that all infoboxes are of the same size ($6 \times 15 = 90$) tokens. For French and German datasets, $fields_per_box$ was set to 15 and $tokens_per_field$ was set to 5. The threshold for $fields_per_box$ for each language was chosen based

on the average number of fields per box in the dataset and then tuned empirically. The target description was restricted to 50 symbols.

4.5.2 Neural network implementation

The actual inputs to the model are provided using placeholders which specify the shape of the inputs to the rest of the computation graph (including batch size). The inputs are fed into the network through the placeholders. Once the placeholders are specified, the computation graph is built in 3 stages: *inference()*, *loss()* and *training()*. The *inference()* procedure specifies the network and a forward pass on the network, the *loss()* adds operations to the graph to compute loss on a given mini-batch of examples (output of *inference()* procedure) and finally the *training()* function takes in the loss and adds operations to compute gradients for updating the parameters of the network. We now proceed to describe each of the above components.

Placeholders

Placeholders specify the dimensions and the datatype of inputs to the model. The following is a list of placeholders followed by a brief description and their corresponding datatype

- encoder_inputs : Provide indexes to lookup embeddings for the input to the model (input sequence for flat model, lower level RNN for hierarchy model) (int)
- dec_input : Indexes to the correct decoder outputs (used for initial epochs of training, in conjunction with the feed previous input) (int)
- field_inputs : Indexes to the field level RNN input (only hierarchy model) (int)
- true_labels : Correct decoder output (used for loss computation) (int)
- weight : Used in conjunction with true labels for ignoring loss computation on non valid time steps (*i.e.* compute loss only on non "<pad>" symbols) (int)
- feed_previous : Choose if the output from the current time-step of the decoder needs to be fed to the next time step or not. (boolean)

Note that even though we use word embeddings to represent inputs, the encoder and decoder inputs to the model are provided as indexes into an appropriate dictionary.

TensorFlow provides interfaces for performing lookup from embedding matrices and hence we directly use indexes as inputs.

Inference

The inference procedure defines a forward pass over the network. Algorithm 1 specifies the pseudocode for the flat model.

Algorithm 1 Inference for flat model

```
procedure inference(encoder_inputs, dec_input, feed_prev)
    enc_in = embedding_lookup(encoder_inputs)
    enc_states = encoder_rnn(enc_in)
    scores, attn_wts_values = attention_dec_flat(enc_states, dec_input, feed_prev)
    return scores, attn_wts_values
end procedure
```

The *attention_dec_flat()* procedure runs the decoder RNN and generates the outputs of the model by implementing equations (4.2) and (4.3), *scores* and *attn_wts_values* denote the output of softmax computation (probability distribution over the vocabulary) and attention weights on the input symbols respectively. The pseudo code for the attention decoder is provided in Algorithm 2.

Algorithm 2 Attention decoder flat model

```
procedure attention_dec_flat(enc_states, dec_input, feed_prev)
    scores = []
    attn_wts_values = []
    for t in range(len(dec_input)) do
        for j in range(len(enc_states)) do
            α_{t,j} = attn(t, enc_states_j)          ▷ attention on inputs (eq. (4.2))
        end for
        c_t = Σ_j α_{t,j} enc_states_j                ▷ context computation
        if feed_prev == True then                     ▷ use previous output or decoder input
            s_t = GRU_dec(s_{t-1}, [e(y_{t-1}); c_{t-1}])   ▷ denotes decoder state at time t
        else
            s_t = GRU_dec(s_{t-1}, [e(dec_input_{t-1}); c_{t-1}])
        end if
        y_t = softmax(W_o f(U_dec s_t + L_dec c_t))   ▷ W_o, U_dec and L_dec are model params
        scores.append(y_t)
        attn_wts_values.append(α_t)
    end for
    return scores, attn_wts_values
end procedure
```

Algorithm 3 specifies the pseudocode for the hierarchy model.

Algorithm 3 Inference for hierarchy model

```
procedure inference(encoder_inputs, field_inputs, dec_input, feed_prev)
    enc_in = embedding_lookup(encoder_inputs)
    field_in = embedding_lookup(field_inputs)
    value_states = encoder_rnn(enc_in)
    field_states = field_rnn(field_in; value_states)        ▷ RNN on concatenated
representation specified by Equation (4.5)
    scores, attention_weights_values, attention_weights_fields =
attention_dec_hierarchy (value_states, field_states, dec_input, feed_prev)
    return scores, attention_weights_values, attention_weights_fields
end procedure
```

The *attention_decoder_hierarchy()* procedure specified in Algorithm 5 runs the decoder RNN and generates the outputs of the model by implementing equations (4.6), (4.9), (4.10) and (4.12).

Loss

The loss is computed as the sum of cross entropy loss at each decoder time-step. Algorithm 4 describes the loss computation procedure.

Algorithm 4 Loss computation

```
procedure loss(scores, true_labels, weight)
    loss_op = 0
    for each example_i in batch do
        loss_ex = cross_entropy_loss(example_i, scores_i, true_labels_i, weight_i)
        loss_op += loss_ex
    end for
    loss_op = loss_op/batch_size
    return loss_op
end procedure
```

The procedure *cross_entropy_loss*($example_i, scores_i, true_labels_i, weight_i$) computes a weighted cross entropy loss averaged across decoder time steps for $example_i$. The total loss is then scaled by the batch size to make the hyperparameters invariant to batch size.

Algorithm 5 Attention decoder hierarchy model

procedure *attention_dec_hierarchy(val_states, field_states, dec_input, feed_previous)*
  scores = []
  attention_weights_values = []
  attention_weights_fields = []
  for *t in range(len(dec_input))* **do**
    for *j in range(len(enc_states))* **do**
      $\alpha_{t,j} = attn(t, val_state_j)$ ▷ *attention on values* (eq. (4.6))
    end for
    $c_t^w = \sum_j \alpha_{t,j} val_state_j$ ▷ *lower level context computation*
    for *k in range(len(field_states))* **do**
      $\beta_{t,k} = attn(t, field_state_k)$ ▷ *attention on fields* (eq. (4.9))
    end for
    $c_t^g = \sum_k \beta_{t,k} field_state_j$ ▷ *field level context computation*
    for *l in range(len(enc_states))* **do**
      $\alpha_{t,l}^{'} = attn(\alpha_{t,l}, \beta_t)$ ▷ *combined attention* (eq. (4.10))
    end for
    $c_t^{w'} = \sum_l \alpha_{t,l}^{'} val_state_l$ ▷ *combined context representation*
    if *feed_previous == True* **then** ▷ *use previous output or decoder input*
      $s_t = GRU_{dec}(s_{t-1}, [e(y_{t-1}); c_{t-1}^{w'}; c_{t-1}^g])$ ▷ *denotes decoder state at time t*
    else
      $s_t = GRU_{dec}(s_{t-1}, [e(dec_input_{t-1}); c_{t-1}^{w'}; c_{t-1}^g])$
    end if
    $y_t = \text{softmax}(W_o f(U_{dec} s_t + L_{dec} c_t^{w'} + M_{dec} c_t^g))$ ▷ W_o, U_{dec}, L_{dec} *and* M_{dec} *are model params*
    scores.append(y_t)
    attention_weights_values.append$(\alpha_t^{'})$
    attention_weights_fields.append(β_t)
  end for
  return scores, attention_weights_values, attention_weights_fields
end procedure

Training

The training procedure (Algorithm 6) takes in the output of the loss operation and performs an update of the network parameters. The $optimizer$ object (line no. 2) provided by TensorFlow implements Adam optimization.

Algorithm 6 Training

1: **procedure** *training(loss, learning_rate)*
2: $\quad$ *optimizer* $= tf.train.AdamOptimizer(learning_rate)$
3: $\quad$ *train_op* $= optimizer.minimize(loss)$
4: $\quad$ *return train_op*
5: **end procedure**

4.5.3 Post processing

Our model allows for copying tokens from the input via a copy mechanism. In particular, the copy mechanism looks for the specific positions where the model generates "<unk>" tokens and copies the relevant words from the input infobox with the highest attention score for the corresponding time step. The attention weights returned by the inference procedure (*attn_wts_values*) specified in Algorithm 1 (flat model) and (*attention_weights_values*) Algorithm 3 (hierarchy model) are saved and used in the post processing step.

We implemented our models in TensorFlow version 0.12 [2] and conducted our experiments on a cluster of NVidia K80 GPUs. We observed that the flat model took around 1.5 hours per epoch and the hierarchy model took around 3 hours per epoch for training. We now proceed to describe our experimental setup.

4.6 Experiments

We tuned the hyperparameters of all the models using a validation set. For all the models, we experimented with GRU state sizes of 128, 256 and 512. We used the same state size for the encoder and decoder GRUs (simply to reduce the hyperparameter

[2] Code available at https://github.com/PrekshaNema25/StructuredData_To_Descriptions

search space.). Similarly, for the hierarchical model, we used the same state size for the GRU cells at both levels of the encoder. The total number of unique words in the corpus is around 400K (this includes the words in the infobox and the target description). Of these we retained only the top 20K words in our vocabulary (same as Lebret *et al.* (2016)). We choose the same size for vocabulary for German and French datasets as well. We initialized the embeddings of these words with pre-trained 300 dimensional Glove embeddings (Pennington *et al.* (2014)). Based on results on the validation set, we found that fine-tuning the word embeddings does not give any benefits so for our final models we did not fine tune the word embeddings. We used Adam optimizer (Kingma and Ba (2015)) with a learning rate of 0.0004, $\beta_1 = 0.9$ and $\beta_2 = 0.999$. We trained the model for a maximum of 20 epochs and used early stopping with the patience set to 5 epochs.

We report the performance of our models using BLEU, NIST and ROUGE metrics and compare the performance of the models both with and without copy mechanism and observe that our models outperform both the baselines. For Lebret *et al.* (2016) we directly take the mean results reported in the original paper. Mei *et al.* (2016*b*) was proposed for the WEATHERGOV and ROBOCUP that have smaller vocabulary (350 words), and hence they did not incorporate copy mechanism for dealing with out of vocabulary words. It is non trivial to extend their model to incorporate a copy mechanism and hence we only report results for their original model. We start with the performance on the English dataset and then discuss the model performance on German and French datasets respectively. Finally, we end the discussion with the performance on the WEATHERGOV dataset.

4.6.1 English

We compare the performance of our models on the English dataset against both the baselines (Lebret *et al.* (2016) and Mei *et al.* (2016*b*)) and Table 4.2 presents the results from our models.

We notice that both the flat model and the hierarchy model outperform the baseline models. Further, the hierarchy model performs the best among all other models. We

Model	No-copy			Copy		
	BLEU-4	**NIST-4**	**ROUGE-4**	**BLEU-4**	**NIST-4**	**ROUGE-4**
Lebret *et al.* (2016)	4.2	1.41	1.48	34.7	7.98	25.8
Mei *et al.* (2016*b*)	35.1	7.27	30.9	–	–	–
Flat model	34.22	7.51	29.75	38.2	8.47	34.28
Hierarchy model	**36.65**	**8.00**	**32.75**	**41.22**	**8.96**	**38.71**

Table 4.2: Comparison of different models on the English WIKIBIO dataset

now present some generated samples from the model [3]

- **Reference:** Vicente Mijares Saldivar (born June 14, 1954 in Gómez Palacio, Durango, Mexico) is an undefeated Mexican professional boxer in the Lightweight division.
- **Flat model:** vicente mijares saldivar (born june 14 , 1954 in gómez palacio , durango) is a mexican boxer in the lightweight division.
- **Hierarchy model:** vicente mijares saldivar (born june 14 , 1954 in gómez palacio , durango , mexico) is a mexican professional boxer in the lightweight division.

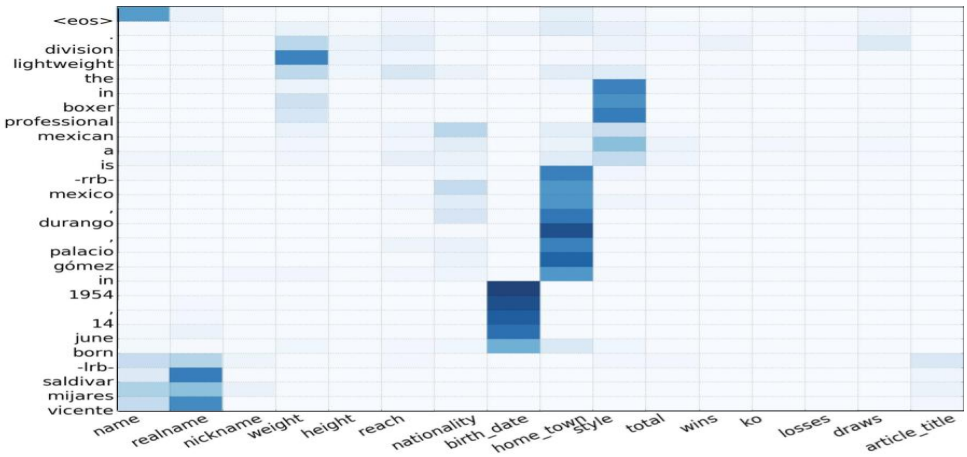

Figure 4.3: Vicente Mijares - Attention Visualization

We notice that both the models generate grammatically correct descriptions and the hierarchy model end up describing some more details of the person. Further, the attention visualization suggests that the model learns to focus on appropriate fields in the infobox while generating words in the description, for example the attention weights are focused on the "birth_date" field when describing the date of birth and "birth_place" when describing the place of birth.

- **Reference:** Professor Masayoshi Tomizuka is a professor in Control Theory in Department of Mechanical Engineering, University of California, Berkeley.

[3] We present post-processed results with -lrb- replaced with '(' and -rrb- replaced with ')', end of sentence(<eos>) symbol removed and trailing space between last word and '.' removed

- **Flat model:** masayoshi tomizuka (born 31 march 1946) is a japanese scientist and professor emeritus at the university of california , berkeley.
- **Hierarchy model:** masayoshi tomizuka (born march 31 , 1946) is a japanese theoretical physicist.

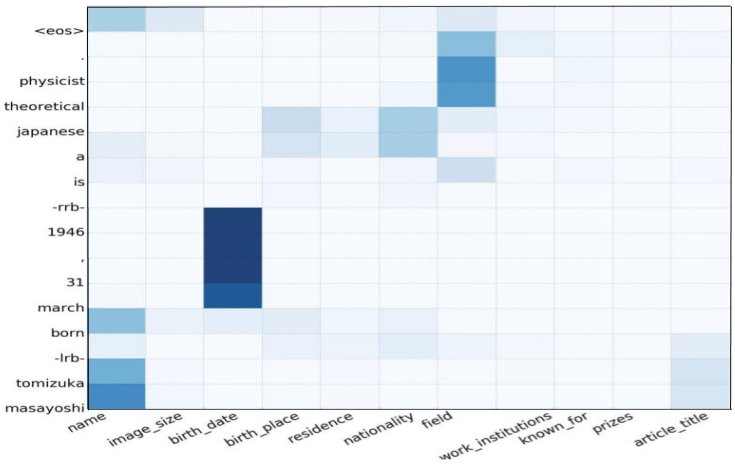

Figure 4.4: Masayoshi Tomizuka - Attention Visualization

In this example, we notice that the flat model generates a description that contains information about the university affiliation of the professor. The hierarchical model gets the profession incorrect but the description is still coherent. Further, we notice that the model has attention weights focussed on appropriate fields while generating relevant words of the description text.

We now mention 3 more sets of examples generated by the models.

- **Reference:** Walter Clarence Henderson (28 February 1891 – 20 September 1968) was a Progressive Conservative party member of the Canadian House of Commons.
- **Flat model:** walter clarence " bert " henderson (february 28 , 1891 – september 20 , 1968) was a canadian politician.
- **Hierarchy model:** walter clarence henderson (28 february 1891 – 20 september 1968) was a progressive conservative party member of the canadian house of commons.

- **Reference:** Arlindo Gomes Semedo (born 17 November 1977), known as Cafú, is a Cape Verdean professional footballer who plays for F.C. Maia Lidador as a forward.
- **Flat model:** arlindo gomes semedo (born 17 november 1977) is a portuguese retired footballer who played as a forward.

- **Hierarchy model:** arlindo gomes semedo (born 17 november 1977) , known as cafú , is a cape verdean professional footballer who plays for portuguese club freamunde as a forward.

- **Reference:** Jordan López Pérez (born 13 November 1986) is a Gibraltarian footballer who currently plays for FC Olmympique 13 in the Gibraltar Second Division, and represented the Gibraltar national team in UEFA competition from 2013 until 2017, as a goalkeeper.
- **Flat model:** jordan perez (born november 13 , 1986) is an american soccer player who plays as a goalkeeper for the 1.85 wolves.
- **Hierarchy model:** jordan perez (born 13 november 1986) is a gibraltarian footballer who currently plays for gibraltar in the gibraltar premier league.

In all the above examples, we notice that the models generate coherent descriptions. At a few places however, the models make errors with the nationality. This could be because multiple countries are mentioned in the infobox and the model is unable to distinguish nationality.

We now proceed to describe the results from experiments on French and German datasets respectively.

4.6.2 French

For the French dataset, we compare the performance of our model with the model presented by Mei *et al.* (2016*b*). Since the code for the model by Lebret *et al.* (2016) was not made publicly available, we do not report the performance of their model on the French dataset [4]. We notice that the flat model does better compared to the hierarchical model based on the BLEU and NIST metrics, and the hierarchy model performs better on the ROUGE metric (Table 4.3). The scores themselves are small compared to the performance of the models on the English dataset. One of the reasons for this could be the signficantly smaller training data available. Further, our qualitative observations from the dataset suggest that French descriptions have less consistent structure compared to the English descriptions.

[4] We tried implementing the model from Lebret *et al.* (2016) but could not exactly replicate the results for the English dataset.

Model	No-copy			Copy		
	BLEU-4	**NIST-4**	**ROUGE-4**	**BLEU-4**	**NIST-4**	**ROUGE-4**
Mei *et al.* (2016*b*)	10.4	2.51	7.81	–	–	–
Flat model	**12.77**	**3.32**	10.27	**14.5**	**3.02**	12.22
Hierarchy model	12.3	3.13	**10.6**	13.8	2.86	**12.37**

Table 4.3: Comparison of different models on the French WIKIBIO dataset

4.6.3 German

Similar to the French dataset, we compare the performance of our model on the German dataset with that of the model presented by Mei *et al.* (2016*b*). Further we do not report the performance of Lebret *et al.* (2016) on the German dataset (Table 4.4). Here we note that the hierarchical model performs the best according to all the 3 metrics. However, the overall numbers are smaller compared to the English dataset. The size of German dataset is 7.5% of the English dataset and has 32.29 tokens on average and hence the performance is relatively poor.

Model	No-copy			Copy		
	BLEU-4	**NIST-4**	**ROUGE-4**	**BLEU-4**	**NIST-4**	**ROUGE-4**
Mei *et al.* (2016*b*)	9.3	2.23	5.85	–	–	–
Flat model	16.98	3.45	12.12	17.05	3.09	12.16
Hierarchy model	**17.76**	**3.60**	**12.98**	**20.38**	**3.43**	**14.89**

Table 4.4: Comparison of different models on the German WIKIBIO dataset

4.6.4 WEATHERGOV

We also evaluate our model on the WEATHERGOV dataset. As a baseline, we compare our model with the work of Mei *et al.* (2016*b*). Note that since the vocabulary in the dataset is limited (350 words) we do not use the copy mechanism. We observe that our models outperform the baseline model (Table 4.5). We notice that the flat and hierarchy model have almost similar performance on the WEATHERGOV dataset. This is primarily because the dataset contains examples from one restricted domain (weather forecasts), has a very small vocabulary (350 words) and has very consistent patterns in the descriptions.

Model	BLEU-4	cBLEU-4
Mei *et al.* (2016*b*)	61.8	71.2
Flat model	**82.5**	88.8
Hierarchy	82.2	**88.9**

Table 4.5: Comparison of different models on the WEATHERGOV dataset

4.7 Summary

In this chapter we discussed our proposed models for biography generation. We described the flat model and the hierarchy model and modelling choices involved in each. We then discussed the experimental results for the performance of the models on English, French and German datasets. The performance of the models as measured by the evaluation metrics indicate that the modelling assumptions (flat and hierachical model) are reasonable and is further validated by some example generated sentences that were presented. In the next chapter, we discuss the conclusions drawn from this thesis and point to directions for future research.

Chapter 5

Conclusions and Future work

In this thesis, we consider the problem of Natural Language Generation from structured data. In particular, we target a specific problem of generating biographical descriptions of people from different domains (sports, politics, arts, *etc.*) based on fact tables describing them. We conduct a study of relevant work from the area of NLG in structured data and propose models for biography generation. Unlike other NLG tasks such as Machine translation, dialog systems *etc.*, the input has specific structure which can be exploited for building models. In particular, we adapt the neural encode-attend-decode (seq2seq) model for our task and propose models that exploit the hierarchical organization of facts in the input fact tables (infoboxes). Our experimental results demonstrate the validity of our modelling choices. These models can be used as a starting point to tackle other problems in NLG from structured data.

An interesting extension of this work is to build models to generate descriptions of infoboxes in multiple languages. In the context of wikipedia infoboxes, we notice that in many cases, information about a person is is not fully specified in one language. If our models could take inputs from infoboxes from multiple languages (if available) we could potentially generate better descriptions. Consider the sample infoboxes (English and French) shown in Figure 5.1 and consider the English description for Leonardo da Vinci below:

"Leonardo di ser Piero da Vinci (15 April 1452 – 2 May 1519), more commonly Leonardo da Vinci or simply Leonardo, was an Italian Renaissance polymath whose areas of interest included invention, painting, sculpting, architecture, science, music, mathematics, engineering, literature, anatomy, geology, astronomy, botany, writing, history, and cartography."

We notice that the English infobox actually does not contain much information however, the description in English contains a lot of facts which indeed do appear in the

Born	Leonardo di ser Piero da Vinci 15 April 1452 Vinci, Republic of Florence (present-day Italy)
Died	2 May 1519 (aged 67) Amboise, Kingdom of France
Known for	Art, science
Works	*Mona Lisa* *The Last Supper* *The Vitruvian Man* *Lady with an Ermine*
Movement	High Renaissance
Signature	

Nom de naissance	Leonardo di ser Piero da Vinci
Activités	Peintre, ingénieur, astronome, philosophe, anatomiste, mathématicien, sculpteur, polymathe, architecte, ingénieur civil, diplomate, inventeur, compositeur, poète, musicien, scientifique, physiologiste
Maître	Andrea del Verrocchio
Élève	Salai, Francesco Melzi...
Lieux de travail	Milan, Amboise, Rome, Florence
Mouvement	Haute Renaissance
Mécènes	Laurent de Médicis, Ludovic Sforza, François Ier...
Père	Piero da Vinci

Figure 5.1: Leonarda da Vinci English and French infobox

French infobox. If models could exploit information across languages, we could generate more informative descriptions. Such models can use combined representation for infoboxes across languages which can then be used to generate descriptions in multiple languages. Such systems can be used to automatically generate descriptions in resource poor languages.

The ideas presented in this thesis could also be used for solving similar NLG problems. Some interesting directions include

- Product descriptions from product catalog tables
  E-commerce sites usually have a large catalog of products and these product typically have a tabulated descriptions of relevant attributes. An NLG system similar to ours can be used to generate short natural langauge descriptions which could make it easy for users to make product purchase decisions.

- Product FAQ generation from product catalogs
  Similar to the setup above, customers on e-commerce websites usually consult FAQs while making purchase decisions. It is helpful then to have more elaborate questions covering the product. It is not practical to manually generate questions and an automated system for generating questions is very desirable. Training such models would require us to curate relevant datasets.

- Hotel descriptions/questions from information on Travel websites
  Travel websites contain tabulated information pertaining hotels/resorts *etc.* along with short textual descriptions. As is the case with product purchases on e-commerce sites, booking decisions can be made easier if relevant textual descriptions are available. NLG systems for generating such descriptions are of practical and commercial relevance.

In conclusion, we present a set of techniques for approaching NLG from structured data problems and demonstrate that our techniques outpeform related baseline models in the specific problem that we consider and we point to promising avenues for pursuing future work.

Bibliography

1. **Angeli, G.**, **P. Liang**, and **D. Klein**, A simple domain-independent probabilistic approach to generation. *In Proceedings of the 2010 Conference on Empirical Methods in Natural Language Processing, EMNLP 2010, 9-11 October 2010, MIT Stata Center, Massachusetts, USA, A meeting of SIGDAT, a Special Interest Group of the ACL*. ACL, 2010. URL `http://www.aclweb.org/anthology/D10-1049`.

2. **Bahdanau, D.**, **K. Cho**, and **Y. Bengio**, Neural machine translation by jointly learning to align and translate. *In Proceedings of the International Conference on Learning Representations (ICLR)*. 2014.

3. **Barzilay, R.** and **M. Lapata**, Collective content selection for concept-to-text generation. *In Proceedings of the Conference on Human Language Technology and Empirical Methods in Natural Language Processing*, HLT '05. Association for Computational Linguistics, Stroudsburg, PA, USA, 2005. URL `http://dx.doi.org/10.3115/1220575.1220617`.

4. **Barzilay, R.** and **M. Lapata**, Aggregation via set partitioning for natural language generation. *In* **R. C. Moore**, **J. A. Bilmes**, **J. Chu-Carroll**, and **M. Sanderson** (eds.), *Human Language Technology Conference of the North American Chapter of the Association of Computational Linguistics, Proceedings, June 4-9, 2006, New York, New York, USA*. The Association for Computational Linguistics, 2006. URL `http://aclweb.org/anthology/N/N06/N06-1046.pdf`.

5. **Barzilay, R.** and **L. Lee**, Catching the drift: Probabilistic content models, with applications to generation and summarization. *In* **J. Hirschberg**, **S. T. Dumais**, **D. Marcu**, and **S. Roukos** (eds.), *Human Language Technology Conference of the North American Chapter of the Association for Computational Linguistics, HLT-NAACL 2004, Boston, Massachusetts, USA, May 2-7, 2004*. The Association for Computational Linguistics, 2004. URL `http://aclweb.org/anthology/N/N04/N04-1015.pdf`.

6. **Belz, A.** (2008). Automatic generation of weather forecast texts using comprehensive probabilistic generation-space models. *Nat. Lang. Eng.*, **14**(4), 431–455. ISSN 1351-3249. URL `http://dx.doi.org/10.1017/S1351324907004664`.

7. **Belz, A.** and **E. Kow**, System building cost vs. output quality in data-to-text generation. *In Proceedings of the 12th European Workshop on Natural Language Generation*, ENLG '09. Association for Computational Linguistics, Stroudsburg, PA, USA, 2009. URL `http://dl.acm.org/citation.cfm?id=1610195.1610198`.

8. **Chen, D. L.** and **R. J. Mooney**, Learning to sportscast: A test of grounded language acquisition. *In Proceedings of the 25th International Conference on Machine Learning (ICML)*. Helsinki, Finland, 2008. URL `http://www.cs.utexas.edu/users/ai-lab/?chen:icml08`.

9. **Cho, K.**, **B. van Merrienboer**, **Ç. Gülçehre**, **D. Bahdanau**, **F. Bougares**, **H. Schwenk**, and **Y. Bengio**, Learning phrase representations using RNN encoder-decoder for statistical machine translation. *In* **A. Moschitti**, **B. Pang**, and **W. Daelemans** (eds.), *Proceedings of the 2014 Conference on Empirical Methods in Natural Language Processing, EMNLP 2014, October 25-29, 2014, Doha, Qatar, A meeting of SIGDAT, a Special Interest Group of the ACL*. ACL, 2014. URL `http://aclweb.org/anthology/D/D14/D14-1179.pdf`.

10. **Chomsky, N.**, *The Logical Structure of Linguistic Theory*. Springer US, 1975. ISBN 9780306307607. URL `https://books.google.co.in/books?id=1D66ktXOITAC`.

11. **Chopra, S.**, **M. Auli**, and **A. M. Rush**, Abstractive sentence summarization with attentive recurrent neural networks. *In Proceedings of the 2016 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies*. Association for Computational Linguistics, San Diego, California, 2016. URL `http://www.aclweb.org/anthology/N16-1012`.

12. **Cybenko, G.** (1989). Approximation by superpositions of a sigmoidal function. *Mathematics of Control, Signals, and Systems (MCSS)*, **2**(4), 303–314.

13. **Dale, R.**, **S. Geldof**, and **J.-P. Prost**, Coral: Using natural language generation for navigational assistance. *In Proceedings of the 26th Australasian Computer Science Conference*. 2003.

14. **Doddington, G.**, Automatic evaluation of machine translation quality using n-gram co-occurrence statistics. *In Proceedings of ARPA Workshop on Human Language Technology*. 2002.

15. **Duboue, P. A.** and **K. R. McKeown**, Content planner construction via evolutionary algorithms and a corpus-based fitness function. *In In Proceedings of the 2 nd International Natural Language Generation Conference (INLG'02*. 2002.

16. **Galanis, D.** and **I. Androutsopoulos**, Generating multilingual descriptions from linguistically annotated owl ontologies: the naturalowl system. *In Proceedings of the 11th European Workshop on Natural Language Generation, ENLG 07*. 2007.

17. **Goodfellow, I.**, **Y. Bengio**, and **A. Courville**, *Deep Learning*. MIT Press, 2016. `http://www.deeplearningbook.org`.

18. **Green, N.**, Generation of biomedical arguments for lay readers. *In Proceedings of the Fourth International Natural Language Generation Conference*, INLG '06. Association for Computational Linguistics, Stroudsburg, PA, USA, 2006. ISBN 1-932432-72-8. URL `http://dl.acm.org/citation.cfm?id=1706269.1706292`.

19. **Hochreiter, S.** and **J. Schmidhuber** (1997). Long short-term memory. *Neural Comput.*, **9**(8), 1735–1780. ISSN 0899-7667. URL `http://dx.doi.org/10.1162/neco.1997.9.8.1735`.

20. **Hornik, K.**, **M. Stinchcombe**, and **H. White** (1989). Multilayer feedforward networks are universal approximators. *Neural networks*, **2**(5), 359–366.

21. **Kalchbrenner, N.** and **P. Blunsom**, Recurrent continuous translation models. *In Proceedings of the 2013 Conference on Empirical Methods in Natural Language Processing, EMNLP 2013, 18-21 October 2013, Grand Hyatt Seattle, Seattle, Washington, USA, A meeting of SIGDAT, a Special Interest Group of the ACL*. ACL, 2013. URL `http://aclweb.org/anthology/D/D13/D13-1176.pdf`.

22. **Kingma, D. P.** and **J. L. Ba**, Adam: A method for stochastic optimization. *In Proceedings of the International Conference on Learning Representations (ICLR)*. 2015.

23. **Konstas, I.** and **M. Lapata**, Unsupervised concept-to-text generation with hypergraphs. *In Proceedings of the 2012 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies*, NAACL HLT '12. Association for Computational Linguistics, Stroudsburg, PA, USA, 2012. ISBN 978-1-937284-20-6. URL `http://dl.acm.org/citation.cfm?id=2382029.2382151`.

24. **Langkilde, I.** and **K. Knight**, Generation that exploits corpus-based statistical knowledge. *In Proceedings of the 36th Annual Meeting of the Association for Computational Linguistics and 17th International Conference on Computational Linguistics - Volume 1*, ACL '98. Association for Computational Linguistics, Stroudsburg, PA, USA, 1998. URL `https://doi.org/10.3115/980845.980963`.

25. **Lebret, R.** and **R. Collobert**, Word embeddings through hellinger pca. *In in Proceedings of the 14th Conference of the European Chapter of the Association for Computational Linguistics*. 2014. URL `http://www.aclweb.org/anthology/E/E14/E14-1051.pdf`.

26. **Lebret, R.**, **D. Grangier**, and **M. Auli**, Neural text generation from structured data with application to the biography domain. *In* **J. Su**, **X. Carreras**, and **K. Duh** (eds.), *Proceedings of the 2016 Conference on Empirical Methods in Natural Language Processing, EMNLP 2016, Austin, Texas, USA, November 1-4, 2016*. The Association for Computational Linguistics, 2016. URL `http://aclweb.org/anthology/D/D16/D16-1128.pdf`.

27. **Liang, P.**, **M. I. Jordan**, and **D. Klein**, Learning semantic correspondences with less supervision. *In Proceedings of the Joint Conference of the 47th Annual Meeting of the ACL and the 4th International Joint Conference on Natural Language Processing of the AFNLP: Volume 1 - Volume 1*, ACL '09. Association for Computational Linguistics, Stroudsburg, PA, USA, 2009. ISBN 978-1-932432-45-9. URL `http://dl.acm.org/citation.cfm?id=1687878.1687893`.

28. **Lin, C. Y.**, Rouge: A package for automatic evaluation of summaries. *In Proceedings of the Workshop on Text Summarization Branches Out (WAS 2004)*. 2004.

29. **Lu, W.** and **H. T. Ng**, A probabilistic forest-to-string model for language generation from typed lambda calculus expressions. *In Proceedings of the 2011 Conference on Empirical Methods in Natural Language Processing, EMNLP 2011, 27-31 July 2011, John McIntyre Conference Centre, Edinburgh, UK, A meeting of SIGDAT, a Special Interest Group of the ACL*. ACL, 2011. URL `http://www.aclweb.org/anthology/D11-1149`.

30. **Luong, T.**, **I. Sutskever**, **Q. Le**, **O. Vinyals**, and **W. Zaremba**, Addressing the rare word problem in neural machine translation. *In Proceedings of the 53rd Annual Meeting of the Association for Computational Linguistics and the 7th International Joint Conference on Natural Language Processing (Volume 1: Long Papers)*. Association for Computational Linguistics, Beijing, China, 2015. URL `http://www.aclweb.org/anthology/P15-1002`.

31. **Mairesse, F.** and **M. A. Walker** (2011). Controlling user perceptions of linguistic style: Trainable generation of personality traits. *Computational Linguistics*.

32. **Manning, C. D.** and **H. Schütze**, *Foundations of Statistical Natural Language Processing*. MIT Press, Cambridge, MA, 1999. ISBN 978-0-262-13360-9.

33. **Mei, H.**, **M. Bansal**, and **M. R. Walter**, Listen, attend, and walk: Neural mapping of navigational instructions to action sequences. *In Proceedings of AAAI*. 2016*a*.

34. **Mei, H.**, **M. Bansal**, and **M. R. Walter**, What to talk about and how? selective generation using lstms with coarse-to-fine alignment. *In Proceedings of the 2016 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies*. Association for Computational Linguistics, San Diego, California, 2016*b*. URL `http://www.aclweb.org/anthology/N16-1086`.

35. **Mikolov, T.**, **I. Sutskever**, **K. Chen**, **G. Corrado**, and **J. Dean**, Distributed representations of words and phrases and their compositionality. *In Neural and Information Processing System (NIPS)*. 2013. URL `https://papers.nips.cc/paper/5021-distributed-representations-of-words-and-phrases-and-their-comp` `pdf`.

36. **Nallapati, R.**, **B. Zhou**, **C. N. dos Santos**, **Ç. Gülçehre**, and **B. Xiang**, Abstractive text summarization using sequence-to-sequence rnns and beyond. *In Proceedings of the 20th SIGNLL Conference on Computational Natural Language Learning, CoNLL 2016, Berlin, Germany, August 11-12, 2016*. 2016. URL `http://aclweb.org/anthology/K/K16/K16-1028.pdf`.

37. **Nema, P.**, **M. M. Khapra**, **A. Laha**, and **B. Ravindran**, Diversity driven attention model for query-based abstractive summarization. *In* **R. Barzilay** and **M. Kan** (eds.), *Proceedings of the 55th Annual Meeting of the Association for Computational Linguistics, ACL 2017, Vancouver, Canada, July 30 - August 4, Volume 1: Long Papers*. Association for Computational Linguistics, 2017. URL `https://doi.org/10.18653/v1/P17-1098`.

38. **Papineni, K.**, **S. Roukos**, **T. Ward**, and **W.-J. Zhu**, Bleu: A method for automatic evaluation of machine translation. *In Proceedings of the 40th Annual Meeting on Association for Computational Linguistics*, ACL '02. Association for Computational Linguistics, Stroudsburg, PA, USA, 2002. URL `https://doi.org/10.3115/1073083.1073135`.

39. **Pascanu, R.**, **T. Mikolov**, and **Y. Bengio**, On the difficulty of training recurrent neural networks. *In* **S. Dasgupta** and **D. McAllester** (eds.), *Proceedings of the 30th International Conference on Machine Learning*, volume 28 of *Proceedings of Machine Learning Research*. PMLR, Atlanta, Georgia, USA, 2013. URL `http://proceedings.mlr.press/v28/pascanu13.html`.

40. **Pennington, J.**, **R. Socher**, and **C. D. Manning**, Glove: Global vectors for word representation. *In Empirical Methods in Natural Language Processing (EMNLP)*. 2014. URL http://www.aclweb.org/anthology/D14-1162.

41. **Ratnaparkhi, A.** (2002). Trainable approaches to surface natural language generation and their application to conversational dialog systems. *Computer Speech & Language*, **16**(3), 435 – 455. ISSN 0885-2308. URL http://www.sciencedirect.com/science/article/pii/S0885230802000256. Spoken Language Generation.

42. **Reiter, E.** and **R. Dale**, *Building Natural Language Generation Systems*. Studies in Natural Language Processing. Cambridge University Press, 2000. ISBN 9780521620369. URL https://books.google.co.in/books?id=qnWQU9C8bDkC.

43. **Reiter, E.**, **S. Sripada**, **J. Hunter**, **J. Yu**, and **I. Davy** (2005). Choosing words in computer-generated weather forecasts. *Artif. Intell.*, **167**(1-2), 137–169. ISSN 0004-3702. URL http://dx.doi.org/10.1016/j.artint.2005.06.006.

44. **Ruder, S.** (2016). An overview of gradient descent optimization algorithms. *CoRR*, **abs/1609.04747**. URL http://arxiv.org/abs/1609.04747.

45. **Rumelhart, D. E.**, **G. E. Hinton**, and **R. J. Williams** (1986). Learning representations by back-propagating errors. *nature*, **323**(6088), 533.

46. **Rush, A. M.**, **S. Chopra**, and **J. Weston**, A neural attention model for abstractive sentence summarization. *In Proceedings of the 2015 Conference on Empirical Methods in Natural Language Processing*. Association for Computational Linguistics, Lisbon, Portugal, 2015. URL http://aclweb.org/anthology/D15-1044.

47. **Schuster, M.**, **K. K. Paliwal**, and **A. General** (1997). Bidirectional recurrent neural networks. *IEEE Transactions on Signal Processing*.

48. **Serban, I. V.**, **A. Sordoni**, **Y. Bengio**, **A. Courville**, and **J. Pineau**, Building end-to-end dialogue systems using generative hierarchical neural network models. *In Proceedings of the Thirtieth AAAI Conference on Artificial Intelligence*, AAAI'16. AAAI Press, 2016. URL http://dl.acm.org/citation.cfm?id=3016387.3016435.

49. **Soricut, R.** and **D. Marcu**, Stochastic language generation using widl-expressions and its application in machine translation and summarization. *In* **N. Calzolari**, **C. Cardie**, and **P. Isabelle** (eds.), *ACL 2006, 21st International Conference on Computational Linguistics and 44th Annual Meeting of the Association for Computational Linguistics, Proceedings of the Conference, Sydney, Australia, 17-21 July 2006*. The Association for Computer Linguistics, 2006. URL http://aclweb.org/anthology/P06-1139.

50. **Sutskever, I.**, **O. Vinyals**, and **Q. V. Le**, Sequence to sequence learning with neural networks. *In* **Z. Ghahramani**, **M. Welling**, **C. Cortes**, **N. D. Lawrence**, and **K. Q. Weinberger** (eds.), *Advances in Neural Information Processing Systems 27*. Curran Associates, Inc., 2014, 3104–3112. URL http://papers.nips.cc/paper/5346-sequence-to-sequence-learning-with-neural-networks.pdf.

51. **Turing, A. M.**, Computing machinery and intelligence. *In Mind*, volume 59. 1950.

52. **Turner, R.**, **Y. Sripada**, and **E. Reiter**, Generating approximate geographic descriptions. *In Proceedings of the 12th European Workshop on Natural Language Generation*, ENLG '09. Association for Computational Linguistics, Stroudsburg, PA, USA, 2009. URL `http://dl.acm.org/citation.cfm?id=1610195.1610202`.

53. **Yang, Z.**, **D. Yang**, **C. Dyer**, **X. He**, **A. Smola**, and **E. Hovy**, Hierarchical attention networks for document classification. *In Proceedings of the 2016 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies*. Association for Computational Linguistics, San Diego, California, 2016. URL `http://www.aclweb.org/anthology/N16-1174`.

54. **Yao, K.**, **G. Zweig**, and **B. Peng** (2015). Attention with intention for a neural network conversation model. *CoRR*, **abs/1510.08565**. URL `http://arxiv.org/abs/1510.08565`.

LIST OF PAPERS BASED ON THESIS

1. Preksha Nema*, Shreyas Shetty M*, Parag Jain*, Anirban Laha, Karthik Sankaranarayanan, Mitesh M. Khapra, "*Generating Descriptions from Structured Data Using a Bifocal Attention Mechanism and Gated Orthogonalization*", *To appear in the Proceedings of the Sixteenth Annual Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies NAACL-HLT-2018, New Orleans, Louisiana, USA, June 1-6*
   (- equal contribution)*